

Rapport de Projet PWEB

"Uber-Cueillette"

-Yosri Dkhil
-Med Amine Aslouj



Plan

ARCHITECTURE DU PROJET UBER-CUEILLETTE

ORGANISATION EN MODULES SÉPARÉS :

AGRICULTEUR/ → GESTION DES OFFRES, CANDIDATS, NOTATIONS

OUVRIER/ → RECHERCHE DE MISSIONS, CANDIDATURES, CHANTIERS

TRAITEMENT/ → LOGIQUE BACKEND (AJOUT, SUPPRESSION, RÉPONSE)

CONFIG/ + BD/ → CONNEXION BDD ET STRUCTURE SQL

CSS/, JS/, IMAGES/ → INTERFACE UTILISATEUR

```
plan
1  uber-cueillette/
2  |  racine/ (fichiers publics)
3  |  |  index.html
4  |  |  login.php
5  |  |  register.php
6  |  |  logout.php
7  |  agriculteur/
8  |  |  dashboard.php
9  |  |  ajouter-offre.php
10 |  |  mes-offres.php
11 |  |  postulants.php
12 |  |  noter-ouvriers.php
13 |  |  profil.php
14 |  |  profil-ouvrier.php
15 |  |  modifier-offre.php
16 |  ouvrier/
17 |  |  dashboard.php
18 |  |  offres-disponibles.php
19 |  |  mes-candidatures.php
20 |  |  mes-chantiers.php
21 |  |  profil.php
22 |  config/
23 |  |  database.php
24 |  |  test.php
25 |  traitement/
26 |  |  ajouter-offre.php
27 |  |  supprimer-condidateur.php
28 |  |  repondre-condidateur.php
29 |  css/
30 |  |  style.css
31 |  js/
32 |  |  script.js
```

```
33 |  images/
34 |  |  logo.png
35 |  |  default-profile.jpg
36 |  |  olive.png
37 |  |  agrumes.png
38 |  |  tomates.png
39 |  |  raisin.png
40 |  |  hero-agriculture1.png
41 |  bd/
42 |  |  uber_cueillette.sql
```

database.php+test.php

```
<?php
/**
 * config/database.php
 * Configuration de la connexion à la base de données
 */

$host = 'localhost';
$dbname = 'uber_cueillette';
$username = 'root';
$password = '';

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch(PDOException $e) {
    die("Erreur de connexion : " . $e->getMessage());
}
?>
```

```
<?php
require_once(__DIR__ . '/database.php');
echo "✅ test.php fonctionne !";
?>
```

DATABASE.PHP :

**GÉRER LA CONNEXION À LA BASE DE DONNÉES
MYSQL**

TEST.PHP:

**VÉRIFIER RAPIDEMENT QUE PHP ET L'INCLUSION
FONCTIONNENT**

registre.php

```
<?php
/**
 * registre.php - Page d'inscription
 * Permet aux agriculteurs et ouvriers de créer un compte
 */
require_once('../config/database.php');
session_start();
// Si déjà connecté, redirection
if (isset($_SESSION['user_id'])) {
    if ($_SESSION['user_type'] == 'agriculteur') {
        header('Location: ../agriculteur/dashboard.php');
    } else {
        header('Location: ../ouvrier/dashboard.php');
    }
    exit();
}
$erreurs = [];
$succes = '';
```

1. EN-TÊTE DU FICHIER (COMMENTAIRE DESCRIPTIF)
2. CONNEXION À LA BASE DE DONNÉES
3. DÉMARRAGE DE LA SESSION
4. VÉRIFICATION : UTILISATEUR DÉJÀ CONNECTÉ
5. REDIRECTION SELON LE TYPE D'UTILISATEUR
6. INITIALISATION DU TABLEAU D'ERREURS
7. INITIALISATION DE LA VARIABLE DE SUCCÈS

registre.php

```
$type = $_GET['type'] ?? 'agriculteur';
// Traitement du formulaire
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $type = $_POST['type'] ?? 'agriculteur';
    // Champs communs
    $nom = trim($_POST['nom'] ?? '');
    $prenom = trim($_POST['prenom'] ?? '');
    $cin = trim($_POST['cin'] ?? '');
    $email = trim($_POST['email'] ?? '');
    $pseudo = trim($_POST['pseudo'] ?? '');
    $password = $_POST['password'] ?? '';
    $confirm_password = $_POST['confirm_password'] ?? '';
    // Validations communes
    if (empty($nom)) $erreurs[] = "Le nom est requis";
    if (empty($prenom)) $erreurs[] = "Le prénom est requis";
    if (empty($cin)) {$erreurs[] = "Le CIN est requis";} elseif (!preg_match('/^[0-9]{8}$/ ', $cin)) {
        $erreurs[] = "Le CIN doit contenir 8 chiffres";}
    if (empty($email)) {
        $erreurs[] = "L'email est requis";} elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $erreurs[] = "Email invalide";}
    if (empty($pseudo)) {
        $erreurs[] = "Le pseudo est requis";
    } elseif (!preg_match('/^[A-Za-z]+$/', $pseudo)) {
        $erreurs[] = "Le pseudo ne doit contenir que des lettres";
    }
    if (empty($password)) {
        $erreurs[] = "Le mot de passe est requis";
    } elseif (strlen($password) < 8) {
        $erreurs[] = "Le mot de passe doit contenir au moins 8 caractères";
    } elseif (!preg_match('/[$#]$/ ', $password)) {
        $erreurs[] = "Le mot de passe doit se terminer par $ ou #";
    }
    if ($password != $confirm_password) {
        $erreurs[] = "Les mots de passe ne correspondent pas";
    }
}
```

8. RÉCUPÉRATION DU TYPE D'UTILISATEUR (GET)
9. DÉTECTION DE LA SOUMISSION DU FORMULAIRE (POST)
10. RÉCUPÉRATION DU TYPE D'UTILISATEUR (POST)
11. RÉCUPÉRATION DES CHAMPS COMMUNS DU FORMULAIRE
12. VALIDATION DU NOM
13. VALIDATION DU PRÉNOM
14. VALIDATION DU CIN (8 CHIFFRES)
15. VALIDATION DE L'EMAIL (FORMAT VALIDE)
16. VALIDATION DU PSEUDO (LETTRES UNIQUEMENT)
17. VALIDATION DU MOT DE PASSE (8 CARACTÈRES MINIMUM)
18. VALIDATION DU MOT DE PASSE (FINIR PAR \$ OU #)
19. VALIDATION DE LA CONFIRMATION DU MOT DE PASSE

registre.php

```
// Vérifier si le pseudo ou email existe déjà
if (empty($erreurs)) {
    try {
        // Vérifier dans agriculteur
        $sql = "SELECT idagriculteur FROM uber_cueilletteagriculteur WHERE pseudo = ? OR email = ? OR CIN = ?";
        $stmt = $pdo->prepare($sql);
        $stmt->execute([$pseudo, $email, $cin]);
        if ($stmt->fetch()) {
            $erreurs[] = "Ce CIN, pseudo ou email est déjà utilisé";
        }
    }

    if (empty($erreurs)) {
        $sql = "SELECT idouvrier FROM uber_cueilletteouvrier WHERE pseudo = ? OR email = ? OR CIN = ?";
        $stmt = $pdo->prepare($sql);
        $stmt->execute([$pseudo, $email, $cin]);
        if ($stmt->fetch()) {
            $erreurs[] = "Ce CIN, pseudo ou email est déjà utilisé";
        }
    }
} catch (PDOException $e) {
    $erreurs[] = "Erreur lors de la vérification";
}
```

- 20. VÉRIFICATION : ABSENCE D'ERREURS DE VALIDATION
- 21. DÉBUT DU BLOC TRY (GESTION DES EXCEPTIONS PDO)
- 22. VÉRIFICATION DES DOUBLONS DANS LA TABLE AGRICULTEUR (PSEUDO, EMAIL, CIN)
- 23. EXÉCUTION DE LA REQUÊTE DE VÉRIFICATION AGRICULTEUR
- 24. AJOUT D'ERREUR SI DOUBLON TROUVÉ (CIN, PSEUDO OU EMAIL DÉJÀ UTILISÉ)
- 25. VÉRIFICATION DES DOUBLONS DANS LA TABLE OUVRIER (PSEUDO, EMAIL, CIN)
- 26. EXÉCUTION DE LA REQUÊTE DE VÉRIFICATION OUVRIER
- 27. AJOUT D'ERREUR SI DOUBLON TROUVÉ (CIN, PSEUDO OU EMAIL DÉJÀ UTILISÉ)
- 28. CAPTURE D'EXCEPTION PDO (ERREUR DE BASE DE DONNÉES)

registre.php

```
if (empty($erreurs)) {
    try {if ($type == 'agriculteur') {$adresse = trim($_POST['adresse'] ?? '');
        if (empty($adresse)) {
            $erreurs[] = "L'adresse est requise";
        } else {
            $sql = "INSERT INTO uber_cueilletteagriculteur (nom, prenom, CIN, email, adresse, pseudo, password)
                VALUES (?, ?, ?, ?, ?, ?, ?)";
            $stmt = $pdo->prepare($sql);
            // ✅ CORRECTION : Pas de hachage, mot de passe en clair
            $stmt->execute([$nom, $prenom, $cin, $email, $adresse, $pseudo, $password]);
            $succes = "✅ Inscription réussie ! Vous pouvez maintenant vous connecter.";
        } } else {
            $description = trim($_POST['description'] ?? '');
            $photo = null;
            if (isset($_FILES['photo']) && $_FILES['photo']['error'] == 0) {
                $maxSize = 2 * 1024 * 1024;
                if ($_FILES['photo']['size'] > $maxSize) {
                    $erreurs[] = "La photo ne doit pas dépasser 2Mo";
                } else {
                    $typePhoto = $_FILES['photo']['type'];
                    if (!in_array($typePhoto, ['image/jpeg', 'image/png'])) {
                        $erreurs[] = "Format accepté : JPG, PNG";
                    } else {
                        $photo = file_get_contents($_FILES['photo']['tmp_name']);
                    }
                }
            }
            if (empty($erreurs)) {
                $sql = "INSERT INTO uber_cueilletteouvrier (nom, prenom, CIN, email, photo, description, pseudo, password)
                    VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
                $stmt = $pdo->prepare($sql);
                // ✅ CORRECTION : Pas de hachage, mot de passe en clair
                $stmt->execute([$nom, $prenom, $cin, $email, $photo, $description, $pseudo, $password]);
                $succes = "✅ Inscription réussie ! Vous pouvez maintenant vous connecter.";
            }
        }
    }
}
```

```
catch (PDOException $e) {
    if ($e->errorInfo[1] == 1062) {
        $erreurs[] = "❌ Ce CIN, email ou pseudo est déjà utilisé.";
    } else {
        $erreurs[] = "❌ Erreur lors de l'inscription";
    }
}
```

VÉRIFICATION ET INSERTION

29. VÉRIFICATION : AUCUNE ERREUR AVANT INSERTION

30. DÉBUT DU BLOC TRY

CAS AGRICULTEUR

31. TYPE = AGRICULTEUR → RÉCUPÉRATION + VALIDATION DE L'ADRESSE

32. INSERT DANS TABLE AGRICULTEUR (PRÉPARATION + EXÉCUTION)

33. MESSAGE DE SUCCÈS

CAS OUVRIER

34. TYPE = OUVRIER → RÉCUPÉRATION DE LA DESCRIPTION

35. VÉRIFICATION PHOTO UPLOADÉE (TAILLE MAX 2MO, FORMATS JPG/PNG)

36. LECTURE DE LA PHOTO EN BINAIRE

37. INSERT DANS TABLE OUVRIER (PRÉPARATION + EXÉCUTION)

38. MESSAGE DE SUCCÈS

GESTION DES ERREURS PDO

39. CAPTURE DE L'EXCEPTION PDO

40. CODE 1062 (DOUBLON) → MESSAGE SPÉCIFIQUE / SINON MESSAGE GÉNÉRIQUE

Login.php

```
<?php
// login.php - Page de connexion
require_once('../config/database.php');
session_start();
// Si déjà connecté, redirection
if (isset($_SESSION['user_id'])) {
    if ($_SESSION['user_type'] == 'agriculteur') {
        header('Location: ../agriculteur/dashboard.php');
    } else {header('Location: ../ouvrier/dashboard.php');}
    exit();
}
$erreur = '';
// Traitement du formulaire
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $username = trim($_POST['username'] ?? '');
    $password = $_POST['password'] ?? '';

    if (!empty($username) && !empty($password)) {
        // Vérifier agriculteur
        $sql = "SELECT * FROM uber_cueillette_agriculteur WHERE pseudo = ?";
        $stmt = $pdo->prepare($sql);
        $stmt->execute([$username]);
        $user = $stmt->fetch();

        if ($user && $password == $user['password']) {
            $_SESSION['user_id'] = $user['id_agriculteur'];
            $_SESSION['user_nom'] = $user['nom'];
            $_SESSION['user_prenom'] = $user['prenom'];
            $_SESSION['user_pseudo'] = $user['pseudo'];
            $_SESSION['user_type'] = 'agriculteur';
            header('Location: ../agriculteur/dashboard.php');
            exit();
        }
        // Vérifier ouvrier
        $sql = "SELECT * FROM uber_cueillette_ouvrier WHERE pseudo = ?";
        $stmt = $pdo->prepare($sql);
        $stmt->execute([$username]);
        $user = $stmt->fetch();
    }
}
```

1. CONNEXION À LA BASE ET DÉMARRAGE DE SESSION
2. VÉRIFICATION : UTILISATEUR DÉJÀ CONNECTÉ
3. INITIALISATION DE LA VARIABLE D'ERREUR
4. TRAITEMENT DU FORMULAIRE (MÉTHODE POST)
5. VÉRIFICATION DES CHAMPS VIDES
6. RECHERCHE DANS LA TABLE AGRICULTEUR
7. VÉRIFICATION ET CONNEXION DE L'AGRICULTEUR
8. RECHERCHE DANS LA TABLE OUVRIER

Login.php

```
if ($user && $password == $user['password']) {  
    $_SESSION['user_id'] = $user['id_ouvrier'];  
    $_SESSION['user_nom'] = $user['nom'];  
    $_SESSION['user_prenom'] = $user['prenom'];  
    $_SESSION['user_pseudo'] = $user['pseudo'];  
    $_SESSION['user_type'] = 'ouvrier';  
    header('Location: ../ouvrier/dashboard.php');  
    exit();  
}  
$erreur = "Pseudo ou mot de passe incorrect";  
} else {  
    $erreur = "Veuillez remplir tous les champs";  
}
```

9. Vérification et connexion de l'ouvrier

10. Message d'erreur générique (identifiants incorrects)

11. Message d'erreur (champs vides)

logout.php

```
<?php
// logout.php - Déconnexion
session_start();
session_destroy();
header('Location: index.html');
exit();
?>
```

1. Démarrage de la session
2. Destruction de la session
(suppression des données)
3. Redirection vers la page
d'accueil (index.html)
4. Arrêt du script (exit)

validation.js

```
// =====  
// 1. VARIABLES GLOBALES (utilisées partout)  
// =====  
let nomApplication = "Uber-Cueillette";  
let versionApplication = 1.0;  
let estDeploye = true;
```

1. VARIABLES GLOBALES

2. FONCTIONS DE VALIDATION DES FORMATS

validerFormat(valeur, type)

Vérifie si un champ respecte le bon format (CIN, pseudo, mot de passe)

```
// Cette fonction vérifie si un champ respecte le bon format  
// Elle est utilisée dans la page registre.php  
function validerFormat(valeur, type) {  
  
    if (type === 'cin') {  
        // CIN : 8 chiffres exactement  
        if (valeur.length !== 8) return false;  
  
        for (let i = 0; i < 8; i++) {  
            let c = valeur.charAt(i);  
            if (c < '0' || c > '9') return false;  
        }  
        return true;  
    }  
  
    if (type === 'pseudo') {  
        // Pseudo : uniquement des lettres  
        for (let i = 0; i < valeur.length; i++) {  
            let c = valeur.charAt(i);  
            let estMinuscule = (c >= 'a' && c <= 'z');  
            let estMajuscule = (c >= 'A' && c <= 'Z');  
  
            if (!estMinuscule && !estMajuscule) return false;  
        }  
        return true;  
    }  
  
    if (type === 'password') {  
        // Mot de passe : 8+ caractères, finit par $ ou #  
        if (valeur.length < 8) return false;  
  
        let dernier = valeur.charAt(valeur.length - 1);  
        return (dernier === '$' || dernier === '#');  
    }  
  
    return false; // Type inconnu  
}
```

validation.js

```
function showError(inputId, message) {  
  // 1. Chercher l'endroit où afficher l'erreur  
  let errorElement = document.getElementById(inputId + '-error');  
  // 2. Si cet endroit existe  
  if (errorElement) {  
    // Mettre le message avec une icône d'erreur  
    errorElement.innerHTML = '<i class="fas fa-exclamation-circle"></i> ' + message;  
    // Mettre le texte en rouge  
    errorElement.style.color = '#e74c3c';  
    // Taille du texte  
    errorElement.style.fontSize = '12px';  
    // Espacement au-dessus  
    errorElement.style.marginTop = '5px';  
    // Espacement en-dessous  
    errorElement.style.marginBottom = '5px';  
    // Afficher l'élément-----  
    errorElement.style.display = 'block';  
  }  
  // 3. Chercher le champ qui a l'erreur  
  let input = document.getElementById(inputId);  
  // 4. Si le champ existe  
  if (input) {  
    // Ajouter une classe "error" pour le CSS  
    input.classList.add('error');  
  
    // Mettre la bordure en rouge  
    input.style.borderColor = '#e74c3c';  
  
    // Mettre la bordure plus épaisse  
    input.style.borderWidth = '2px';  
  }  
}
```

3. FONCTIONS D'AFFICHAGE DES ERREURS

showError(inputId, message):
Affiche un message d'erreur sous un champ et le met en rouge

validation.js

```
// Cette fonction efface le message d'erreur d'un champ
// Utilisée dans : login.php, registre.php
function clearError(inputId) {
    // 1. Chercher l'endroit où l'erreur était affichée
    let errorElement = document.getElementById(inputId + '-error');
    // 2. Si cet endroit existe
    if (errorElement) {
        // Vider le message
        errorElement.innerHTML = '';

        // Cacher l'élément
        errorElement.style.display = 'none';
    }
    // 3. Chercher le champ
    let input = document.getElementById(inputId);
    // 4. Si le champ existe
    if (input) {
        // Enlever la classe "error"
        input.classList.remove('error');

        // Remettre la bordure normale
        input.style.borderColor = '';

        // Remettre la bordure fine
        input.style.borderWidth = '1px';
    }
}
```

clearError(inputId)

Efface le message d'erreur d'un champ et restaure sa bordure normale

validation.js

```
// Cette fonction efface toutes les erreurs d'un formulaire
// Utilisée dans : registre.php (avant de valider)
function clearErrors(formId) {
    // 1. Chercher le formulaire
    let form = document.getElementById(formId);
    // 2. Si le formulaire n'existe pas, on arrête
    if (!form) {
        return;
    }
    // 3. Chercher tous les messages d'erreur dans le formulaire
    let errorMessages = form.querySelectorAll('.error-message');
    // 4. Pour chaque message d'erreur
    for (let i = 0; i < errorMessages.length; i++) {
        // Vider le texte
        errorMessages[i].innerHTML = '';

        // Cacher l'élément
        errorMessages[i].style.display = 'none';
    }
    // 5. Chercher tous les champs du formulaire
    let inputs = form.querySelectorAll('input, textarea, select');
    // 6. Pour chaque champ
    for (let i = 0; i < inputs.length; i++) {
        // Enlever la classe "error"
        inputs[i].classList.remove('error');

        // Remettre la bordure normale
        inputs[i].style.borderColor = '';

        // Remettre la bordure fine
        inputs[i].style.borderWidth = '1px';
    }
}
```

clearErrors(formId)

**Efface toutes les erreurs d'un
formulaire complet**

validation.js

```
// =====  
// 4. MODE SOMBRE/CLAIR (utilisé dans toutes les pages)  
// =====  
  
// Quand la page est chargée  
document.addEventListener('DOMContentLoaded', function() {  
    // Initialiser le thème (sombre/clair)  
    initTheme();  
  
    // Initialiser les formulaires  
    initForms();  
  
    // Ajouter la classe active aux liens  
    ajouterClassesActive();  
  
    // Initialiser la validation en direct  
    initLiveValidation();  
});
```

4. FONCTIONS DU MODE SOMBRE/CLAIR

validation.js

```
function initTheme() {  
  // Chercher le bouton de changement de theme  
  let themeToggle = document.getElementById('theme-toggle');  
  // Si le bouton n'existe pas, on arrête  
  if (!themeToggle) {  
    return;  
  }  
  // Récupérer l'élément HTML principal  
  let htmlElement = document.documentElement;  
  // Récupérer le thème sauvegardé dans le navigateur  
  let savedTheme = localStorage.getItem('theme');  
  // Si pas de thème sauvegardé, on met "light" (clair)  
  if (savedTheme === null) {  
    savedTheme = 'light';  
  }  
  // Appliquer le thème à la page  
  htmlElement.setAttribute('data-theme', savedTheme);  
  // Mettre à jour l'icône du bouton  
  updateThemeIcon(savedTheme);  
  // Quand on clique sur le bouton  
  themeToggle.addEventListener('click', function() {  
    // Récupérer le thème actuel  
    let currentTheme = htmlElement.getAttribute('data-theme');  
  
    // Changer le thème  
    let newTheme;  
    if (currentTheme === 'light') {  
      newTheme = 'dark';  
    } else {  
      newTheme = 'light';  
    }  
    // Appliquer le nouveau thème  
    htmlElement.setAttribute('data-theme', newTheme);  
    // Sauvegarder dans le navigateur  
    localStorage.setItem('theme', newTheme);  
    // Mettre à jour l'icône  
    updateThemeIcon(newTheme);  
  });  
}
```

initTheme()

**Initialise le mode sombre/clair et
gère le bouton de changement**

validation.js

```
// Cette fonction met à jour l'icône du bouton de thème
function updateThemeIcon(theme) {

    // Chercher le bouton
    let themeToggle = document.getElementById('theme-toggle');

    // Si le bouton n'existe pas, on arrête
    if (!themeToggle) {
        return;
    }

    // Chercher l'icône à l'intérieur du bouton
    let icon = themeToggle.querySelector('i');

    // Si l'icône existe
    if (icon) {
        // Si le thème est clair, on met une lune
        if (theme === 'light') {
            icon.className = 'fas fa-moon';
        }
        // Si le thème est sombre, on met un soleil
        else {
            icon.className = 'fas fa-sun';
        }
    }
}
```

updateThemeIcon(theme)

Met à jour l'icône du bouton (lune pour clair, soleil pour sombre)

validation.js

```
// =====  
// 5. VALIDATION EN DIRECT (utilisée dans login.php et registre.php)  
// =====  
  
// Cette fonction active la validation pendant qu'on tape  
function initLiveValidation() {  
    // =====  
    // Pour la page login.php  
    // =====  
    let loginForm = document.getElementById('login-form');  
    if (loginForm) {  
        let usernameInput = document.getElementById('username');  
        let passwordInput = document.getElementById('password');  
        // Validation du pseudo en direct  
        if (usernameInput) {  
            usernameInput.addEventListener('input', function() {  
                validateField('username', this.value, 'pseudo');  
            });  
            usernameInput.addEventListener('blur', function() {  
                validateField('username', this.value, 'pseudo');  
            });  
        }  
        // Validation du mot de passe en direct  
        if (passwordInput) {  
            passwordInput.addEventListener('input', function() {  
                validateField('password', this.value, 'password');  
            });  
            passwordInput.addEventListener('blur', function() {  
                validateField('password', this.value, 'password');  
            });  
        }  
    }  
}
```

5. FONCTIONS DE VALIDATION EN DIRECT

initLiveValidation():

Active la validation des champs pendant que l'utilisateur tape (login et registre)

validation.js

```
// =====  
// Pour la page registre.php - Formulaire Agriculteur  
// =====  
let agriculteurForm = document.getElementById('agriculteurForm');  
if (agriculteurForm) {  
    // Validation du nom  
    let nomAgri = document.getElementById('nom_agri');  
    if (nomAgri) {  
        nomAgri.addEventListener('input', function() {  
            validateField('nom_agri', this.value, 'text');  
        });  
        nomAgri.addEventListener('blur', function() {  
            validateField('nom_agri', this.value, 'text');  
        });  
    }  
  
    // Validation du prénom  
    let prenomAgri = document.getElementById('prenom_agri');  
    if (prenomAgri) {  
        prenomAgri.addEventListener('input', function() {  
            validateField('prenom_agri', this.value, 'text');  
        });  
        prenomAgri.addEventListener('blur', function() {  
            validateField('prenom_agri', this.value, 'text');  
        });  
    }  
}
```

```
let ouvrierForm = document.getElementById('ouvrierForm');  
  
if (ouvrierForm) {  
    // Validation du nom  
    let nomOuv = document.getElementById('nom_ouv');  
    if (nomOuv) {  
        nomOuv.addEventListener('input', function() {  
            validateField('nom_ouv', this.value, 'text');  
        });  
        nomOuv.addEventListener('blur', function() {  
            validateField('nom_ouv', this.value, 'text');  
        });  
    }  
  
    // Validation du prénom  
    let prenomOuv = document.getElementById('prenom_ouv');  
    if (prenomOuv) {  
        prenomOuv.addEventListener('input', function() {  
            validateField('prenom_ouv', this.value, 'text');  
        });  
        prenomOuv.addEventListener('blur', function() {  
            validateField('prenom_ouv', this.value, 'text');  
        });  
    }  
  
    // Validation du CIN  
    let cinOuv = document.getElementById('cin_ouv');  
    if (cinOuv) {  
        cinOuv.addEventListener('input', function() {  
            validateField('cin_ouv', this.value, 'cin');  
        });  
    }  
}
```

validation.js

```
function validateField(fieldId, valeur, type) {
  clearError(fieldId);
  // =====
  // 2. VERIFIER SI LE CHAMP EST VIDE
  // =====
  if (valeur === "" || valeur === null) {
    showError(fieldId, "Ce champ est obligatoire");
    return false; // On arrête tout de suite
  }
  // Enlever les espaces au début et à la fin
  valeur = valeur.trim();
  // =====
  // 3. VERIFIER SELON LE TYPE DE CHAMP
  // =====
  // -----
  // CAS 1 : EMAIL
  // -----
  if (type === "email") {
    // Vérifier 1 : doit contenir @
    if (valeur.indexOf("@") === -1) {
      showError(fieldId, "L'email doit contenir un @");
      return false;
    }
    // Vérifier 2 : doit contenir . après le @
    let positionArobase = valeur.indexOf("@");
    let partieApresArobase = valeur.substring(positionArobase + 1);
    if (partieApresArobase.indexOf(".") === -1) {
      showError(fieldId, "L'email doit avoir un point après le @ (ex: .com)");
      return false;
    }
    // Vérifier 3 : pas d'espace
    if (valeur.indexOf(" ") !== -1) {
      showError(fieldId, "L'email ne doit pas contenir d'espace");
      return false;
    }
    // Si tout est bon
    return true;
  }
}
```

6. FONCTIONS DE VALIDATION DES CHAMPS

validateField(fieldId, valeur, type):

Vérifie un champ selon son type (email, CIN, pseudo, mot de passe, texte)

validation.js

```
function validateConfirmField(fieldId, valeur, matchId) {  
    // Effacer l'ancienne erreur  
    clearError(fieldId);  
  
    // Vérifier si le champ est vide  
    if (!valeur || valeur.trim() === '') {  
        showError(fieldId, 'Veuillez confirmer votre mot de passe');  
        return false;  
    }  
  
    // Récupérer la valeur du mot de passe original  
    let passwordValue = document.getElementById(matchId).value;  
  
    // Vérifier si les deux mots de passe sont identiques  
    if (valeur !== passwordValue) {  
        showError(fieldId, 'Les mots de passe ne correspondent pas');  
        return false;  
    }  
  
    return true;  
}
```

validateConfirmField(fieldId, valeur, matchId):

**Vérifie que la confirmation du mot de passe
correspond à l'original**

validation.js

```
// Cette fonction vérifie la photo uploadée
function validatePhoto(input) {

  // Effacer l'ancienne erreur
  clearError('photo');

  // Vérifier si une photo a été sélectionnée
  if (!input.files || input.files.length === 0) {
    showError('photo', 'La photo est requise');
    return false;
  }

  // Récupérer le fichier
  let file = input.files[0];

  // Taille maximum : 2 Mo
  let maxSize = 2 * 1024 * 1024;

  // Vérifier la taille
  if (file.size > maxSize) {
    showError('photo', 'La photo ne doit pas dépasser 2Mo');
    return false;
  }

  // Types autorisés
  let allowedTypes = ['image/jpeg', 'image/png'];

  // Vérifier le type de fichier
  if (!allowedTypes.includes(file.type)) {
    showError('photo', 'Format accepté : JPG ou PNG uniquement');
    return false;
  }

  return true;
}
```

validatePhoto(input):

**Vérifie la photo uploadée (taille max 2Mo,
formats JPG/PNG)**

validation.js

```
function initForms() {  
    // Si on est sur la page registre.php  
    if (document.getElementById('agriculteurForm') || document.getElementById('ouvrierForm')) {  
        initRegistrationForms();  
    }  
  
    // Si on est sur la page login.php  
    if (document.getElementById('login-form')) {  
        initLoginForm();  
    }  
}  
  
// =====  
// 8. FONCTIONS POUR LA PAGE D'INSCRIPTION (registre.php)  
// =====  
  
// Cette fonction initialise les formulaires d'inscription  
function initRegistrationForms() {  
    // Récupérer le paramètre "type" dans l'URL  
    let urlParams = new URLSearchParams(window.location.search);  
    let type = urlParams.get('type');  
  
    // Afficher le bon formulaire selon le type  
    if (type === 'ouvrier') {  
        showOuvrierForm();  
    } else {  
        showAgriculteurForm();  
    }  
}
```

7. FONCTIONS D'INITIALISATION DES FORMULAIRES

initForms():

Initialise les formulaires selon la page affichée (registre ou login)

8. FONCTIONS POUR LA PAGE D'INSCRIPTION (registre.php)

initRegistrationForms():

Initialise les formulaires d'inscription et affiche celui correspondant au type dans l'URL

validation.js

```
// Cette fonction affiche le formulaire Agriculteur
function showAgriculteurForm() {

  let agriculteurForm = document.getElementById('agriculteurForm');
  let ouvrierForm = document.getElementById('ouvrierForm');
  let btnAgriculteur = document.getElementById('btnAgriculteur');
  let btnOuvrier = document.getElementById('btnOuvrier');

  // Afficher le formulaire agriculteur
  if (agriculteurForm) {
    agriculteurForm.style.display = 'block';
  }

  // Cacher le formulaire ouvrier
  if (ouvrierForm) {
    ouvrierForm.style.display = 'none';
  }

  // Mettre le bouton agriculteur en actif
  if (btnAgriculteur) {
    btnAgriculteur.classList.add('active');
  }

  // Enlever l'actif du bouton ouvrier
  if (btnOuvrier) {
    btnOuvrier.classList.remove('active');
  }

  // Mettre à jour l'URL (sans recharger la page)
  let url = new URL(window.location);
  url.searchParams.set('type', 'agriculteur');
  window.history.pushState({}, '', url);
  //window.history.pushState(etat, titre, url);
}
```

showAgriculteurForm():

Affiche le formulaire agriculteur et cache celui de l'ouvrier

validation.js

```
// Cette fonction affiche le formulaire Ouvrier
function showOuvrierForm() {

  let agriculteurForm = document.getElementById('agriculteurForm');
  let ouvrierForm = document.getElementById('ouvrierForm');
  let btnAgriculteur = document.getElementById('btnAgriculteur');
  let btnOuvrier = document.getElementById('btnOuvrier');

  // Cacher le formulaire agriculteur
  if (agriculteurForm) {
    agriculteurForm.style.display = 'none';
  }

  // Afficher le formulaire ouvrier
  if (ouvrierForm) {
    ouvrierForm.style.display = 'block';
  }

  // Enlever l'actif du bouton agriculteur
  if (btnAgriculteur) {
    btnAgriculteur.classList.remove('active');
  }

  // Mettre le bouton ouvrier en actif
  if (btnOuvrier) {
    btnOuvrier.classList.add('active');
  }

  // Mettre à jour l'URL (sans recharger la page)
  let url = new URL(window.location);
  url.searchParams.set('type', 'ouvrier');
  window.history.pushState({}, '', url);
}
```

showOuvrierForm():

**Affiche le formulaire ouvrier et
cache celui de l'agriculteur**

validation.js

```
// Cette fonction affiche un aperçu de la photo sélectionnée
function previewPhoto(event) {

    let file = event.target.files[0];

    if (file) {

        // Valider la photo d'abord
        let photoValide = validatePhoto(event.target);

        if (!photoValide) {
            return;
        }

        // Créer un lecteur de fichier
        let reader = new FileReader();

        // Quand le fichier est chargé
        reader.onload = function(e) {

            let preview = document.getElementById('photo-preview');

            if (preview) {
                // Mettre l'image dans la prévisualisation
                preview.src = e.target.result;
                // e.target.result = le contenu du fichier converti en URL de données
                // Afficher la prévisualisation
                preview.classList.add('show');
                preview.style.display = 'block';
            }
        };

        // Lire le fichier
        reader.readAsDataURL(file);
        // Cette méthode lit le fichier et le convertit en URL de données (format data:image/jpeg;base64,...).
    }
}
```

previewPhoto(event):

**Affiche un aperçu de la photo
sélectionnée avant upload**

validation.js

```
function validateAgriculteurForm(event) {
  // Empêcher l'envoi du formulaire pour l'instant
  event.preventDefault();
  // Effacer toutes les erreurs
  clearErrors('agriculteurForm');
  // Récupérer les valeurs des champs
  let nom = document.getElementById('nom_agri').value.trim();
  let prenom = document.getElementById('prenom_agri').value.trim();
  let cin = document.getElementById('cin_agri').value;
  let email = document.getElementById('email_agri').value.trim();
  let adresse = document.getElementById('adresse_agri').value.trim();
  let pseudo = document.getElementById('pseudo_agri').value;
  let password = document.getElementById('password_agri').value;
  let confirmPassword = document.getElementById('confirm_password_agri').value;
  let cgv = document.getElementById('cgv_agri').checked;
  let isValid = true;

  // Valider chaque champ
  if (!validateField('nom_agri', nom, 'text')) {
    isValid = false;
  }

  if (!validateField('prenom_agri', prenom, 'text')) {
    isValid = false;
  }

  if (!validateField('cin_agri', cin, 'cin')) {
    isValid = false;
  }

  if (!validateField('email_agri', email, 'email')) {
    isValid = false;
  }

  if (!validateField('adresse_agri', adresse, 'text')) {
    isValid = false;
  }
}
```

9. VALIDATION DES FORMULAIRES D'INSCRIPTION

validateAgriculteurForm(event):

Valide tous les champs du
formulaire agriculteur avant envoi

validation.js

```
// =====  
function validateOuvrierForm(event) {  
  
    // Empêcher l'envoi du formulaire pour l'instant  
    event.preventDefault();  
  
    // Effacer toutes les erreurs  
    clearErrors('ouvrierForm');  
  
    // Récupérer les valeurs des champs  
    let nom = document.getElementById('nom_ouv').value.trim();  
    let prenom = document.getElementById('prenom_ouv').value.trim();  
    let cin = document.getElementById('cin_ouv').value;  
    let email = document.getElementById('email_ouv').value.trim();  
    let description = document.getElementById('description').value.trim();  
    let pseudo = document.getElementById('pseudo_ouv').value;  
    let password = document.getElementById('password_ouv').value;  
    let confirmPassword = document.getElementById('confirm_password_ouv').value;  
    let cgv = document.getElementById('cgv_ouv').checked;  
    let photo = document.getElementById('photo');  
  
    let isValid = true;  
  
    // Valider chaque champ  
    if (!validateField('nom_ouv', nom, 'text')) {  
        isValid = false;  
    }  
  
    if (!validateField('prenom_ouv', prenom, 'text')) {  
        isValid = false;  
    }  
  
    if (!validateField('cin_ouv', cin, 'cin')) {  
        isValid = false;  
    }  
}
```

validateOuvrierForm(event):

**Valide tous les champs du
formulaire ouvrier avant envoi**

validation.js

```
// Cette fonction initialise la page de connexion
function initLoginForm() {

    let loginForm = document.getElementById('login-form');

    if (!loginForm) {
        return;
    }

    let usernameInput = document.getElementById('username');
    let passwordInput = document.getElementById('password');

    // Quand on tape dans le champ pseudo, on efface l'erreur
    if (usernameInput) {
        usernameInput.addEventListener('input', function() {
            clearError('username');
            hideLoginAlert();
        });
    }

    // Quand on tape dans le champ mot de passe, on efface l'erreur
    if (passwordInput) {
        passwordInput.addEventListener('input', function() {
            clearError('password');
            hideLoginAlert();
        });
    }
}

// Cette fonction cache l'alerte de la page de connexion
function hideLoginAlert() {

    let alert = document.getElementById('login-alert');

    if (alert) {
        alert.style.display = 'none';
    }
}
```

10. FONCTIONS POUR LA PAGE DE CONNEXION (login.php)

initLoginForm():

Initialise la page de connexion
(efface les erreurs en tapant)

hideLoginAlert():

Cache l'alerte de la page de
connexion

validation.js

```
function showLoginAlert(message, type) {  
    let alert = document.getElementById('login-alert');  
  
    if (!alert) {  
        return;  
    }  
  
    // Choisir l'icône selon le type (succès ou erreur)  
    if (type === 'success') {  
        alert.innerHTML = '<i class="fas fa-check-circle"></i> ' + message;  
    } else {  
        alert.innerHTML = '<i class="fas fa-exclamation-circle"></i> ' + message;  
    }  
  
    alert.className = 'alert alert-' + type;  
    alert.style.display = 'block';  
  
    // Cacher l'alerte après 3 secondes  
    setTimeout(function() {alert.style.display = 'none';}, 3000);  
}  
  
// Cette fonction valide le formulaire de connexion  
function validateLoginForm(event) {  
  
    // Effacer toutes les erreurs  
    clearErrors('login-form');  
  
    // Récupérer les valeurs  
    let username = document.getElementById('username').value.trim();  
    let password = document.getElementById('password').value;  
  
    let isValid = true;  
  
    // Validation du pseudo  
    if (!username) {  
        showError('username', 'Pseudo requis');  
        isValid = false;  
    } else {  
        // Vérifier que le pseudo ne contient que des lettres
```

showLoginAlert(message, type):
Affiche une alerte (succès ou erreur) sur la page de connexion

validateLoginForm(event):

Valide le formulaire de connexion (pseudo et mot de passe)

validation.js

```
// Cette fonction ajoute la classe "active" aux liens quand on scrolle
function ajouterClassesActive() {

    // 1. Récupérer les sections de la page
    let section1 = document.getElementById('fonctionnement');
    let section2 = document.getElementById('avantages');
    let section3 = document.getElementById('contact');

    // 2. Mettre les sections dans un tableau
    let toutesLesSections = [section1, section2, section3];
    let sectionsQuiExistent = [];

    // 3. Garder seulement les sections qui sont sur la page
    for (let i = 0; i < toutesLesSections.length; i++) {
        if (toutesLesSections[i] !== null) { // Si la section existe
            sectionsQuiExistent.push(toutesLesSections[i]);
        }
    }

    // 4. Si aucune section n'existe, on arrête
    if (sectionsQuiExistent.length === 0) {
        return; // Sortir de la fonction
    }

    // 5. DÉTECTER LE SCROLL
    window.addEventListener('scroll', function() {

        let sectionVisible = ''; // Pour stocker quelle section est visible

        // 6. Vérifier chaque section
        for (let i = 0; i < sectionsQuiExistent.length; i++) {
            let section = sectionsQuiExistent[i];
            let positionSection = section.offsetTop; // Où commence la section

            // Si l'utilisateur a scrollé jusqu'à la section (ou 200px avant)
            if (window.pageYOffset >= positionSection - 200) {
                sectionVisible = section.getAttribute('id'); // On note l'ID
            }
        }

        // Ici on pourrait ajouter le code pour ajouter la classe "active" aux liens
    });
}
```

ajouterClassesActive():

Ajoute la classe "active" aux liens du menu quand on scrolle vers une section

validation.js

```
// 8. Parcourir chaque lien
for (let i = 0; i < liensMenu.length; i++) {
  // Enlever la classe "active" de tous les liens
  liensMenu[i].classList.remove('active');

  // Récupérer la cible du lien (ex: "#fonctionnement")
  let cible = liensMenu[i].getAttribute('href');

  // Si le lien correspond à la section visible
  if (cible === '#' + sectionVisible) {
    // Ajouter la classe "active" à ce lien
    liensMenu[i].classList.add('active');
  }
}
});
```

```
// Cette fonction gère les liens internes (avec #)
let anchors = document.querySelectorAll('a[href^="#"]');

for (let i = 0; i < anchors.length; i++) {
  anchors[i].addEventListener('click', function(e) {

    let href = this.getAttribute('href');

    // Ignorer les liens vides
    if (href === '#') {
      return;
    }

    let target = document.querySelector(href);

    if (target) {
      // Empêcher le comportement par défaut
      e.preventDefault();
```

```
// Cette fonction gère les liens internes (avec #)
let anchors = document.querySelectorAll('a[href^="#"]');

for (let i = 0; i < anchors.length; i++) {
  anchors[i].addEventListener('click', function(e) {

    let href = this.getAttribute('href');

    // Ignorer les liens vides
    if (href === '#') {
      return;
    }

    let target = document.querySelector(href);

    if (target) {
      // Empêcher le comportement par défaut
      e.preventDefault();

      // Scroller en douceur vers la cible
      target.scrollIntoView({
        behavior: 'smooth',
        block: 'start'
      });
    }
  });
}
```

Agriculteur

Ajouter offre :

CE FICHIER REPRÉSENTE LA PAGE D'AJOUT D'UNE OFFRE AGRICOLE.

Il permet à un agriculteur connecté de remplir un formulaire pour publier une offre de travail (récolte, type de fruit, localisation, prix, etc.) destinée aux ouvriers agricoles.

```
session_start();

if (!isset($_SESSION['user_id']) || $_SESSION['user_type'] !== 'agriculteur') {
    header('Location: ../racine/login.php');
    exit();
}
```

1. Démarrage de session et sécurité :

On démarre la session utilisateur.

On vérifie que :

l'utilisateur est connecté

et qu'il est bien un agriculteur

Sinon → redirection vers la page de login.

But : sécuriser l'accès à la page et empêcher les ouvriers ou visiteurs d'y accéder.

```
// =====
// PARTIE 2: CONNEXION À LA BASE DE DONNÉES
// =====

require_once('../config/database.php');

$idagriculteur = $_SESSION['user_id'];
```

2. Connexion à la base de données:

On inclut le fichier de connexion à la base de données.

Permet d'utiliser \$pdo pour faire des requêtes SQL.

On récupère l'identifiant de l'agriculteur connecté pour associer l'offre à son compte.

```
// =====
// PARTIE 3: RÉCUPÉRER LES MESSAGES D'ERREUR/SUCCÈS
// =====

$erreurs = $_SESSION['erreurs'] ?? [];
$success = $_SESSION['success'] ?? '';

// Effacer les messages après les avoir récupérés
unset($_SESSION['erreurs']);
unset($_SESSION['success']);
```

3. Gestion des messages (erreurs et succès) :

Récupère les messages envoyés après
traitement du formulaire :
erreurs de validation
message de succès

Puis les supprime de la session.

Permet d'afficher un feedback utilisateur.

```
// =====
// PARTIE 4: RÉCUPÉRER LES DONNÉES PRÉCÉDENTES (EN CAS D'ERREUR)
// =====

$old = $_SESSION['old'] ?? [];
unset($_SESSION['old']);
```

4 . Récupération des anciennes données:

Si le formulaire échoue, les
anciennes valeurs sont
conservées.

Cela évite à l'utilisateur de tout
re-saisir.

```
// =====
// PARTIE 5: RÉCUPÉRER LES LISTES POUR LES SELECTS
// =====
// Types de fruits depuis la BDD
$sql = "SELECT * FROM uber_cueillette_type_fruit ORDER BY libelle";
$stmt = $pdo->query($sql);
$types_fruits = $stmt->fetchAll();

// Gouvernorats depuis la BDD
$sql = "SELECT * FROM uber_cueillette_gouvernorat ORDER BY libelle";
$stmt = $pdo->query($sql);
$gouvernorats = $stmt->fetchAll();
// Si les tables sont vides, on utilise des valeurs par défaut
if (empty($types_fruits)) {
    $types_fruits = [
        ['id_type_fruit' => 1, 'libelle' => 'Olives'],
        ['id_type_fruit' => 2, 'libelle' => 'Agrumes'],
        ['id_type_fruit' => 3, 'libelle' => 'Tomates'],
        ['id_type_fruit' => 4, 'libelle' => 'Raisins'],
        ['id_type_fruit' => 5, 'libelle' => 'Pêches'],
        ['id_type_fruit' => 6, 'libelle' => 'Pommes'],
        ['id_type_fruit' => 7, 'libelle' => 'Figues'],
        ['id_type_fruit' => 8, 'libelle' => 'Grenades'],
        ['id_type_fruit' => 9, 'libelle' => 'Autres']
    ];
}
if (empty($gouvernorats)) {
    $gouvernorats = [
        ['id_gouvernorat' => 1, 'libelle' => 'Tunis'],
        ['id_gouvernorat' => 2, 'libelle' => 'Ariana'],
        ['id_gouvernorat' => 3, 'libelle' => 'Ben Arous'],
        ['id_gouvernorat' => 4, 'libelle' => 'Manouba'],
        ['id_gouvernorat' => 5, 'libelle' => 'Nabeul'],
        ['id_gouvernorat' => 6, 'libelle' => 'Zaghouan'],
        ['id_gouvernorat' => 7, 'libelle' => 'Bizerte'],
        ['id_gouvernorat' => 8, 'libelle' => 'Béja'],
        ['id_gouvernorat' => 9, 'libelle' => 'Jendouba'],
        ['id_gouvernorat' => 10, 'libelle' => 'Kef'],
        ['id_gouvernorat' => 11, 'libelle' => 'Siliana'],
        ['id_gouvernorat' => 12, 'libelle' => 'Sousse'],
        ['id_gouvernorat' => 13, 'libelle' => 'Monastir'],
        ['id_gouvernorat' => 14, 'libelle' => 'Mahdia'],
        ['id_gouvernorat' => 15, 'libelle' => 'Sfax'],
        ['id_gouvernorat' => 16, 'libelle' => 'Kairouan'],
        ['id_gouvernorat' => 17, 'libelle' => 'Kasserine'],
        ['id_gouvernorat' => 18, 'libelle' => 'Sidi Bouzid'],
        ['id_gouvernorat' => 19, 'libelle' => 'Gabès'],
        ['id_gouvernorat' => 20, 'libelle' => 'Médenine'],
    ];
}
```

5. Récupération des données dynamiques (BDD)

On récupère les types de fruits depuis la base de données.

Triés par ordre alphabétique (libelle).

On Récupère les régions de Tunisie depuis la base.

Si la base de données est vide, le système utilise des valeurs prédéfinies.

Cela garantit que le formulaire fonctionne toujours.

INTERFACE HTML (STRUCTURE DE LA PAGE):

```
<!-- =====  
PARTIE 6: BARRE DE NAVIGATION  
===== -->  
  
<nav class="navbar">  
  <div class="container">  
    <a href="../racine/index.html" class="logo">  
        
      <span>Uber<span>Cueillette</span></span>  
    </a>  
  
    <ul class="nav-menu">  
      <li><a href="dashboard.php"><i class="fas fa-chart-line"></i> Dashboard</a></li>  
      <li><a href="ajouter-offre.php" class="active"><i class="fas fa-plus-circle"></i> Ajouter offre</a></li>  
      <li><a href="mes-offres.php"><i class="fas fa-list"></i> Mes offres</a></li>  
      <li><a href="profil.php"><i class="fas fa-user"></i> Profil</a></li>  
      <li class="nav-buttons">  
        <a href="../racine/logout.php" class="btn btn-outline">  
          <i class="fas fa-sign-out-alt"></i> Déconnexion  
        </a>  
        <button id="theme-toggle" class="theme-toggle">  
          <i class="fas fa-moon"></i>  
        </button>  
      </li>  
    </ul>  
  </div>  
</nav>
```

6.1 Navigation :

La barre de navigation permet à l'agriculteur de :

accéder au dashboard

ajouter une offre

voir ses offres

modifier son profil

se déconnecter

Elle assure la navigation principale du site.

FORMULAIRE D'AJOUT D'OFFRE

```
<!-- =====
PARTIE 10: FORMULAIRE D'AJOUT
===== -->

<div class="form-container" style="max-width: 800px; margin: 0 auto 50px;">

  <form id="ajoutOffreForm" action="../../traitement/ajouter-offre.php" method="POST">

    <!-- Type de fruit -->
    <div class="form-group">
      <label for="type_fruit">
        <i class="fas fa-apple-alt"></i> Type de fruit
      </label>
      <select id="type_fruit" name="type_fruit">
        <option value="">Sélectionnez un fruit</option>
        <?php foreach($types_fruits as $fruit): ?>
          <option value="<?php echo $fruit['libelle']; ?>"
            <?php echo (isset($old['type_fruit']) && $old['type_fruit'] == $fruit['libelle']) ? 'selected' : ''; ?>>
            <?php echo $fruit['libelle']; ?>
          </option>
        <?php endforeach; ?>
      </select>
      <div class="error-message" id="type_fruit-error"></div>
    </div>

    <!-- Localisation -->
    <div style="display: grid; grid-template-columns: 1fr 2fr; gap: 15px;">

      <div class="form-group">
        <label for="gouvernorat">
          <i class="fas fa-map-marker-alt"></i> Gouvernorat
        </label>
        <select id="gouvernorat" name="gouvernorat">
          <option value="">Sélectionnez</option>
          <?php foreach($gouvernorats as $gouv): ?>
            <option value="<?php echo $gouv['libelle']; ?>"
              <?php echo (isset($old['gouvernorat']) && $old['gouvernorat'] == $gouv['libelle']) ? 'selected' : ''; ?>>
              <?php echo $gouv['libelle']; ?>
            </option>
          <?php endforeach; ?>
        </select>
        <div class="error-message" id="gouvernorat-error"></div>
      </div>
    </div>

  </form>

</div>
```

La partie du formulaire dans ce fichier représente le cœur fonctionnel de la page, car elle permet à l'agriculteur de saisir toutes les informations nécessaires pour créer une offre de travail agricole. Ce formulaire commence par la sélection du type de fruit, dont les options sont chargées dynamiquement depuis la base de données afin de garantir la cohérence des données. Ensuite, l'utilisateur doit choisir le gouvernorat et entrer une adresse précise pour localiser l'exploitation agricole. Le formulaire inclut également la définition de la période de récolte à travers une date de début et une date de fin, ce qui permet de planifier la durée du travail des ouvriers. Une date limite de candidature est aussi demandée afin de contrôler la période durant laquelle les ouvriers peuvent postuler à l'offre.

FORMULAIRE D'AJOUT D'OFFRE

```
<!-- Période -->
<div class="form-section">
  <h3><i class="fas fa-calendar-alt"></i> Période de récolte</h3>

  <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 15px;">

    <div class="form-group">
      <label for="date_debut">Date de début</label>
      <input type="date" id="date_debut" name="date_debut"
        value="<?php echo $old['date_debut'] ?? ''; ?>">
      <div class="error-message" id="date_debut-error"></div>
    </div>

    <div class="form-group">
      <label for="date_fin">Date de fin</label>
      <input type="date" id="date_fin" name="date_fin"
        value="<?php echo $old['date_fin'] ?? ''; ?>">
      <div class="error-message" id="date_fin-error"></div>
    </div>
  </div>
</div>

<!-- Date limite -->
<div class="form-section">
  <h3><i class="fas fa-clock"></i> Date limite</h3>

  <div class="form-group">
    <label for="date_limite">Date limite de postulation</label>
    <input type="date" id="date_limite" name="date_limite"
      value="<?php echo $old['date_limite'] ?? ''; ?>">
    <small style="color: gray; display: block; margin-top: 5px;">
      <i class="fas fa-info-circle"></i> Après cette date, l'offre ne sera plus visible
    </small>
    <div class="error-message" id="date_limite-error"></div>
  </div>
</div>

<!-- Détails de l'offre -->
<div class="form-section">
  <h3><i class="fas fa-chart-bar"></i> Détails de l'offre</h3>

  <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 15px;">
```

Par ailleurs, le formulaire contient des champs relatifs aux détails de l'offre, notamment le nombre d'ouvriers nécessaires et le prix journalier proposé, ce qui permet de définir les conditions économiques et organisationnelles du travail. Une zone de texte est également prévue pour ajouter une description complémentaire, où l'agriculteur peut préciser des informations importantes comme les conditions de travail, l'hébergement ou les repas. Enfin, une case à cocher impose l'acceptation des conditions générales d'utilisation, garantissant ainsi la responsabilité de l'utilisateur sur les informations saisies. L'ensemble de ces champs est structuré de manière à guider l'utilisateur dans la création d'une offre complète, claire et exploitable par les ouvriers agricoles.

JAVASCRIPT (VALIDATION)

```
function validateOffreForm() {  
  
    // Réinitialiser les erreurs  
    let errorDivs = document.querySelectorAll('.error-message');  
    for (let i = 0; i < errorDivs.length; i++) {  
        errorDivs[i].innerHTML = '';  
    }  
  
    let inputs = document.querySelectorAll('input, select, textarea');  
    for (let i = 0; i < inputs.length; i++) {  
        inputs[i].style.borderColor = '';  
    }  
  
    let isValid = true;  
  
    // Récupérer les valeurs  
    let typeFruit = document.getElementById('type_fruit').value;  
    let gouvernorat = document.getElementById('gouvernorat').value;  
    let adresse = document.getElementById('adresse').value.trim();  
    let dateDebut = document.getElementById('date_debut').value;  
    let dateFin = document.getElementById('date_fin').value;  
    let dateLimite = document.getElementById('date_limite').value;  
    let nbOuvriers = document.getElementById('nb_ouvriers').value;  
    let prix = document.getElementById('prix').value;  
    let conditions = document.getElementById('conditions').checked;  
  
    // Validation type fruit  
    if (typeFruit === '') {  
        document.getElementById('type_fruit-error').innerHTML = 'Veuillez sélectionner un type de fruit';  
        document.getElementById('type_fruit').style.borderColor = 'red';  
        isValid = false;  
    }  
  
    // Validation gouvernorat  
    if (gouvernorat === '') {  
        document.getElementById('gouvernorat-error').innerHTML = 'Veuillez sélectionner un gouvernorat';  
        document.getElementById('gouvernorat').style.borderColor = 'red';  
        isValid = false;  
    }  
  
    // Validation adresse  
    if (adresse === '') {  
        document.getElementById('adresse-error').innerHTML = 'L\'adresse est requise';  
        document.getElementById('adresse').style.borderColor = 'red';  
        isValid = false;  
    }  
}
```

La partie de validation dans ce code joue un rôle essentiel pour garantir la qualité et la cohérence des données saisies par l'utilisateur avant l'envoi du formulaire. Elle est implémentée en JavaScript via la fonction `validateOffreForm()`, qui est exécutée lors de la soumission du formulaire. Cette fonction commence par réinitialiser tous les messages d'erreurs et les styles des champs afin de repartir d'un état propre. Ensuite, elle récupère les valeurs saisies par l'utilisateur dans les différents champs du formulaire.

```
    } else if (adresse.length < 5) {  
        document.getElementById('adresse-error').innerHTML = 'L\'adresse doit contenir au moins 5 caractères';  
        document.getElementById('adresse').style.borderColor = 'red';  
        isValid = false;  
    }  
  
    // Validation dates  
    if (dateDebut === '') {  
        document.getElementById('date_debut-error').innerHTML = 'La date de début est requise';  
        document.getElementById('date_debut').style.borderColor = 'red';  
        isValid = false;  
    }  
  
    if (dateFin === '') {  
        document.getElementById('date_fin-error').innerHTML = 'La date de fin est requise';  
        document.getElementById('date_fin').style.borderColor = 'red';  
        isValid = false;  
    }  
  
    if (dateLimite === '') {  
        document.getElementById('date_limite-error').innerHTML = 'La date limite est requise';  
        document.getElementById('date_limite').style.borderColor = 'red';  
        isValid = false;  
    }  
  
    // Validation relations entre dates  
    if (dateDebut && dateFin && dateLimite) {  
        let debut = new Date(dateDebut);  
        let fin = new Date(dateFin);  
        let limite = new Date(dateLimite);  
        let aujourd'hui = new Date();  
        aujourd'hui.setHours(0,0,0,0);  
  
        if (limite < aujourd'hui) {  
            document.getElementById('date_limite-error').innerHTML = 'La date limite doit être dans le futur';  
            document.getElementById('date_limite').style.borderColor = 'red';  
            isValid = false;  
        }  
    }  
}
```

JAVASCRIPT (VALIDATION)

```
} else if (adresse.length < 5) {
    document.getElementById('adresse-error').innerHTML = 'L\'adresse doit contenir au moins 5 caractères';
    document.getElementById('adresse').style.borderColor = 'red';
    isValid = false;
}

// Validation dates
if (dateDebut === '') {
    document.getElementById('date_debut-error').innerHTML = 'La date de début est requise';
    document.getElementById('date_debut').style.borderColor = 'red';
    isValid = false;
}

if (dateFin === '') {
    document.getElementById('date_fin-error').innerHTML = 'La date de fin est requise';
    document.getElementById('date_fin').style.borderColor = 'red';
    isValid = false;
}

if (dateLimite === '') {
    document.getElementById('date_limite-error').innerHTML = 'La date limite est requise';
    document.getElementById('date_limite').style.borderColor = 'red';
    isValid = false;
}

// Validation relations entre dates
if (dateDebut && dateFin && dateLimite) {
    let debut = new Date(dateDebut);
    let fin = new Date(dateFin);
    let limite = new Date(dateLimite);
    let aujourd'hui = new Date();
    aujourd'hui.setHours(0,0,0,0);

    if (limite < aujourd'hui) {
        document.getElementById('date_limite-error').innerHTML = 'La date limite doit être dans le futur';
        document.getElementById('date_limite').style.borderColor = 'red';
        isValid = false;
    }
}
```

La validation vérifie d'abord les champs obligatoires comme le type de fruit, le gouvernorat, l'adresse, les dates, le nombre d'ouvriers, le prix et l'acceptation des conditions générales. Si un champ est vide ou incorrect, un message d'erreur spécifique est affiché et le champ est mis en évidence visuellement. Ensuite, des validations plus avancées sont effectuées sur les dates : la date limite doit être supérieure ou égale à la date actuelle, et la date de fin doit être strictement postérieure à la date de début, afin d'assurer une logique temporelle correcte.

JAVASCRIPT (VALIDATION)

La validation inclut également des contrôles numériques, notamment pour le nombre d'ouvriers qui doit être compris entre 1 et 50, ainsi que pour le prix journalier qui doit se situer dans une plage définie entre 10 et 500 dinars. Enfin, la case des conditions générales est obligatoire et doit être cochée pour permettre la soumission. Si une seule de ces règles n'est pas respectée, la fonction retourne false, empêchant ainsi l'envoi du formulaire au serveur. Cette validation côté client améliore l'expérience utilisateur en détectant rapidement les erreurs et en réduisant les requêtes inutiles vers le serveur.

```
// Validation nombre d'ouvriers
if (nbOuvriers === '') {
    document.getElementById('nb_ouvriers-error').innerHTML = 'Le nombre d\'ouvriers est requis';
    document.getElementById('nb_ouvriers').style.borderColor = 'red';
    isValid = false;
} else if (nbOuvriers < 1 || nbOuvriers > 50) {
    document.getElementById('nb_ouvriers-error').innerHTML = 'Le nombre d\'ouvriers doit être entre 1 et 50';
    document.getElementById('nb_ouvriers').style.borderColor = 'red';
    isValid = false;
}

// Validation prix
if (prix === '') {
    document.getElementById('prix-error').innerHTML = 'Le prix journalier est requis';
    document.getElementById('prix').style.borderColor = 'red';
    isValid = false;
} else if (prix < 10 || prix > 500) {
    document.getElementById('prix-error').innerHTML = 'Le prix doit être entre 10 et 500 DT';
    document.getElementById('prix').style.borderColor = 'red';
    isValid = false;
}

// Validation conditions
if (!conditions) {
    document.getElementById('conditions-error').innerHTML = 'Vous devez accepter les conditions générales';
    isValid = false;
}

return isValid;
```

```
window.onload = function() {

    // Confirmation avant réinitialisation
    const resetBtn = document.querySelector('button[type="reset"]');
    if (resetBtn) {
        resetBtn.addEventListener('click', function(e) {
            if (!confirm('Êtes-vous sûr de vouloir réinitialiser le formulaire ?')) {
                e.preventDefault();
            }
        });
    }
};

// =====
// FONCTION 3: ATTACHER LA VALIDATION AU FORMULAIRE
// =====

document.getElementById('ajoutOffreForm').onsubmit = function() {
    return validateOffreForm();
};
</script>

</body>
</html>
```

Agriculteur

Dashboard :

Le fichier dashboard.php représente la page principale de l'espace agriculteur. Il permet à l'utilisateur de visualiser un résumé global de son activité sur la plateforme. Cette page affiche des statistiques, les offres récentes ainsi que les candidatures reçues. Elle sert donc de tableau de bord central pour gérer toutes les opérations liées aux offres agricoles.

1. Sécurité et vérification de session

Au début du code, une vérification de session est effectuée pour s'assurer que l'utilisateur est bien connecté et qu'il possède le rôle d'agriculteur. Si ces conditions ne sont pas respectées, l'utilisateur est automatiquement redirigé vers la page de connexion. Cette étape garantit la sécurité et empêche l'accès non autorisé au dashboard.

2. Connexion à la base de données

Le fichier de connexion à la base de données est inclus afin de permettre l'exécution des requêtes SQL. L'identifiant de l'agriculteur connecté est récupéré depuis la session pour personnaliser les données affichées. Toutes les informations du dashboard dépendent de cet identifiant.

```
// =====  
// PARTIE 3: RÉCUPÉRER LES INFORMATIONS DE L'AGRICULTEUR  
// =====  
  
$sql = "SELECT * FROM uber_cueilletteagriculteur WHERE idagriculteur = ?";  
$stmt = $pdo->prepare($sql);  
$stmt->execute([$idagriculteur]);  
$agriculteur = $stmt->fetch();  
  
if (!$agriculteur) {  
    session_destroy();  
    header('Location: ../racine/login.php');  
    exit();  
}
```

3. Récupération des informations de l'agriculteur

Une requête est exécutée pour récupérer les informations complètes de l'agriculteur connecté (nom, prénom, email, pseudo). Ces données sont utilisées pour personnaliser l'interface et afficher un message de bienvenue. Si aucun agriculteur n'est trouvé, la session est supprimée et l'utilisateur est redirigé vers la page de connexion.

```
// =====
// PARTIE 4: COMPTER LES OFFRES ACTIVES
// =====

$sql = "SELECT COUNT(*) as total FROM uber_cueillette_offre
        WHERE idagriculteur = ? AND date_limite >= CURDATE()";
$stmt = $pdo->prepare($sql);
$stmt->execute([$idagriculteur]);
$resultat = $stmt->fetch();
$offres_actives = $resultat['total'] ?? 0;
```

4. Calcul des statistiques

Plusieurs requêtes SQL sont utilisées pour calculer les statistiques principales du dashboard. On compte d'abord les offres actives, c'est-à-dire celles dont la date limite n'est pas encore dépassée. Ensuite, le système calcule le nombre de candidatures en attente, en reliant les offres aux candidatures. Il calcule aussi les offres terminées, dont la date de fin est dépassée, ainsi que le total général des offres créées par l'agriculteur.

```
// =====
// PARTIE 5: COMPTER LES CANDIDATURES EN ATTENTE
// =====

$sql = "SELECT COUNT(*) as total FROM uber_cueillette_candidature c
        JOIN uber_cueillette_offre o ON c.id_offre = o.id_offre
        WHERE o.idagriculteur = ? AND c.decision = 'en_attente'";
$stmt = $pdo->prepare($sql);
$stmt->execute([$idagriculteur]);
$resultat = $stmt->fetch();
$candidatures_attente = $resultat['total'] ?? 0;

// =====
// PARTIE 6: COMPTER LES OFFRES TERMINÉES
// =====

$sql = "SELECT COUNT(*) as total FROM uber_cueillette_offre
        WHERE idagriculteur = ? AND date_fin < CURDATE()";
$stmt = $pdo->prepare($sql);
$stmt->execute([$idagriculteur]);
$resultat = $stmt->fetch();
$offres_terminees = $resultat['total'] ?? 0;

// =====
// PARTIE 7: COMPTER LE TOTAL DES OFFRES
// =====

$sql = "SELECT COUNT(*) as total FROM uber_cueillette_offre
        WHERE idagriculteur = ?";
$stmt = $pdo->prepare($sql);
$stmt->execute([$idagriculteur]);
$resultat = $stmt->fetch();
$total_offres = $resultat['total'] ?? 0;

// =====
// PARTIE 8: CALCULER LA NOTE MOYENNE
// =====

$sql = "SELECT AVG(c.note) as moyenne FROM uber_cueillette_candidature c
        JOIN uber_cueillette_offre o ON c.id_offre = o.id_offre
        WHERE o.idagriculteur = ? AND c.note IS NOT NULL";
$stmt = $pdo->prepare($sql);
$stmt->execute([$idagriculteur]);
$resultat = $stmt->fetch();
$note_moyenne = $resultat['moyenne'] ?? 0;
if ($note_moyenne == 0) {
    $note_moyenne = '4.8';
} else {
    $note_moyenne = number_format($note_moyenne, 1);
}
```

```
// =====
// PARTIE 9: RÉCUPÉRER LES 3 DERNIÈRES OFFRES
// =====

$sql = "SELECT o.*, tf.libelle as type_fruit, g.libelle as gouvernorat_libelle,
  (SELECT COUNT(*) FROM uber_cueillette_candidature WHERE id_offre = o.id_offre) as nb_candidatures,
  (SELECT COUNT(*) FROM uber_cueillette_candidature WHERE id_offre = o.id_offre AND decision = 'acceptee') as nb_acceptes
FROM uber_cueillette_offre o
JOIN uber_cueillette_type_fruit tf ON o.id_type_fruit = tf.id_type_fruit
JOIN uber_cueillette_gouvernorat g ON o.id_gouvernorat = g.id_gouvernorat
WHERE o.idagriculteur = ?
ORDER BY o.date_limite DESC
LIMIT 3";
$stmt = $pdo->prepare($sql);
$stmt->execute([$idagriculteur]);
$offres_recentes = $stmt->fetchAll();

// =====
// PARTIE 10: RÉCUPÉRER LES 5 DERNIÈRES CANDIDATURES
// =====

$sql = "SELECT c.*, o.adresse, tf.libelle as type_fruit,
  ouv.nom as ouvrier_nom, ouv.prenom as ouvrier_prenom, ouv.id_ouvrier,
  o.id_offre, o.nombre_ouvriers
FROM uber_cueillette_candidature c
JOIN uber_cueillette_offre o ON c.id_offre = o.id_offre
JOIN uber_cueillette_type_fruit tf ON o.id_type_fruit = tf.id_type_fruit
JOIN uber_cueillette_ouvrier ouv ON c.id_ouvrier = ouv.id_ouvrier
WHERE o.idagriculteur = ?
ORDER BY c.date_candidature DESC
LIMIT 5";
$stmt = $pdo->prepare($sql);
$stmt->execute([$idagriculteur]);
$candidatures_recentes = $stmt->fetchAll();

?>
```

5. Récupération des offres récentes

Une requête SQL permet de récupérer les trois dernières offres créées par l'agriculteur. Ces offres incluent des informations détaillées comme le type de fruit, le gouvernorat, le nombre de candidatures et le nombre d'ouvriers acceptés. Cela permet d'afficher un aperçu rapide des dernières activités.

6. Récupération des candidatures récentes

Le dashboard récupère également les cinq dernières candidatures reçues. Ces données sont obtenues en joignant plusieurs tables (offres, ouvriers, candidatures). Chaque candidature contient des informations sur l'ouvrier, l'offre concernée et le statut de la demande. Cela permet à l'agriculteur de suivre facilement les demandes récentes.

7. Interface utilisateur et navigation

La page contient une barre de navigation permettant d'accéder aux principales fonctionnalités : dashboard, ajout d'offre, gestion des offres et profil utilisateur. Une section d'accueil affiche également un message de bienvenue personnalisé avec le nom de l'agriculteur connecté.

8. Statistiques du tableau de bord

Le dashboard affiche quatre cartes de statistiques principales : les offres actives, les candidatures en attente, les offres terminées et le total des offres. Ces données permettent à l'agriculteur d'avoir une vision rapide et synthétique de son activité sur la plateforme.

9. Affichage des offres récentes

Les offres récentes sont affichées sous forme de cartes. Chaque carte contient les informations essentielles comme le fruit, la localisation, la période, le nombre de candidatures et le prix. Un système de statut indique si l'offre est encore ouverte ou clôturée. Une barre de progression peut également montrer l'évolution des candidatures acceptées.

10. Affichage des candidatures récentes

Les candidatures sont affichées sous forme de tableau. Chaque ligne représente une candidature avec le nom de l'ouvrier, la date, le statut et des actions possibles comme accepter, refuser ou consulter le profil. Cela permet une gestion rapide et efficace des demandes.

```
// =====
// FONCTION 2: FILTRER LES OFFRES
// =====

function filtrerOffres() {

    let statutChoisi = document.getElementById('filtre-statut-offres').value;
    let rechercheTexte = document.getElementById('recherche-offres').value.toLowerCase();
    let triChoisi = document.getElementById('tri-offres').value;

    let offres = document.querySelectorAll('.offre-card');
    let offresVisibles = 0;

    for (let i = 0; i < offres.length; i++) {

        let offre = offres[i];
        let statutOffre = offre.dataset.statut;
        let fruitOffre = offre.dataset.fruit;
        let lieuOffre = offre.dataset.lieu;

        let statutOK = false;
        if (statutChoisi === 'toutes') {
            statutOK = true;
        } else if (statutOffre === statutChoisi) {
            statutOK = true;
        } else {
            statutOK = false;
        }

        let rechercheOK = false;
        if (rechercheTexte === '') {
            rechercheOK = true;
        } else if (fruitOffre.indexOf(rechercheTexte) !== -1) {
            rechercheOK = true;
        } else if (lieuOffre.indexOf(rechercheTexte) !== -1) {
            rechercheOK = true;
        } else {
            rechercheOK = false;
        }

        if (statutOK && rechercheOK) {
            offre.style.display = 'block';
            offresVisibles = offresVisibles + 1;
        } else {
            offre.style.display = 'none';
        }
    }

    document.getElementById('nb-offres-affichees').innerHTML = offresVisibles;
}
```

11. Système de filtrage et recherche

Le dashboard intègre plusieurs filtres pour améliorer l'expérience utilisateur. L'agriculteur peut filtrer les offres par statut, rechercher par mot-clé ou trier selon différents critères. De même, les candidatures peuvent être filtrées par nom, statut ou date. Cela facilite la gestion lorsque le nombre de données est important.

```
        if (triChoisi !== 'date') {
            trierOffres(triChoisi);
        }
    }

// =====
// FONCTION 3: TRIER LES OFFRES
// =====

function trierOffres(critere) {

    let grid = document.getElementById('offres-grid');
    let offres = Array.from(document.querySelectorAll('.offre-card'));

    if (critere === 'candidatures') {
        offres.sort(function(a, b) {
            return b.dataset.candidatures - a.dataset.candidatures;
        });
    } else if (critere === 'prix') {
        offres.sort(function(a, b) {
            return a.dataset.prix - b.dataset.prix;
        });
    }

    for (let i = 0; i < offres.length; i++) {
        grid.appendChild(offres[i]);
    }
}

// =====
```

```
// =====
// FONCTION 5: FILTRER LES CANDIDATURES
// =====

function filtrerCandidatures() {

    let rechercheTexte = document.getElementById('recherche-ouvrier').value.toLowerCase();
    let statutChoisi = document.getElementById('filtre-statut-candidatures').value;
    let filtreDate = document.getElementById('filtre-date-candidatures').value;

    let lignes = document.querySelectorAll('#candidatures-body tr');
    let aujourd'hui = new Date();
    aujourd'hui.setHours(0, 0, 0, 0);

    let debutSemaine = new Date(aujourd'hui);
    debutSemaine.setDate(aujourd'hui.getDate() - aujourd'hui.getDay());

    let debutMois = new Date(aujourd'hui.getFullYear(), aujourd'hui.getMonth(), 1);

    let lignesVisibles = 0;

    for (let i = 0; i < lignes.length; i++) {

        let ligne = lignes[i];
        let nomOuvrier = ligne.dataset.ouvrier;
        let statutLigne = ligne.dataset.statut;
        let timestamp = ligne.dataset.date * 1000;
        let dateLigne = new Date(timestamp);

        let rechercheOK = false;
        if (rechercheTexte === '') {
            rechercheOK = true;
        } else if (nomOuvrier.indexOf(rechercheTexte) !== -1) {
            rechercheOK = true;
        } else {
            rechercheOK = false;
        }

        let statutOK = false;
        if (statutChoisi === 'tous') {
            statutOK = true;
        } else if (statutLigne === statutChoisi) {
            statutOK = true;
        } else {
            statutOK = false;
        }

    }

    document.getElementById('nb-candidatures-affichees').innerHTML = lignesVisibles;
}

// =====
// FONCTION 6: RÉINITIALISER LES FILTRES CANDIDATURES
// =====

function reinitialiserFiltresCandidatures() {

    document.getElementById('recherche-ouvrier').value = '';
    document.getElementById('filtre-statut-candidatures').value = 'tous';
    document.getElementById('filtre-date-candidatures').value = 'toutes';

    filtrerCandidatures();
}
```

11. Système de filtrage et recherche

Le dashboard intègre plusieurs filtres pour améliorer l'expérience utilisateur. L'agriculteur peut filtrer les offres par statut, rechercher par mot-clé ou trier selon différents critères. De même, les candidatures peuvent être filtrées par nom, statut ou date. Cela facilite la gestion lorsque le nombre de données est important.

```
let dateOK = false;
if (filtreDate === 'toutes') {
    dateOK = true;
} else if (filtreDate === 'aujourd'hui') {
    dateOK = dateLigne.toDateString() === aujourd'hui.toDateString();
} else if (filtreDate === 'semaine') {
    dateOK = dateLigne >= debutSemaine;
} else if (filtreDate === 'mois') {
    dateOK = dateLigne >= debutMois;
}

if (rechercheOK && statutOK && dateOK) {
    ligne.style.display = '';
    lignesVisibles = lignesVisibles + 1;
} else {
    ligne.style.display = 'none';
}

document.getElementById('nb-candidatures-affichees').innerHTML = lignesVisibles;
}

// =====
// FONCTION 6: RÉINITIALISER LES FILTRES CANDIDATURES
// =====

function reinitialiserFiltresCandidatures() {

    document.getElementById('recherche-ouvrier').value = '';
    document.getElementById('filtre-statut-candidatures').value = 'tous';
    document.getElementById('filtre-date-candidatures').value = 'toutes';

    filtrerCandidatures();
}
```

Agriculteur

mes_offres :

Cette page permet à l'utilisateur de consulter de manière détaillée ses offres, ainsi que les informations associées telles que le type de fruit, le gouvernorat, la période de récolte, le nombre d'ouvriers demandés, les candidatures reçues et le statut de chaque offre (ouverte, clôturée ou terminée).

1 : Sécurisation et session

Le fichier commence par l'ouverture de la session avec `session_start()`. Ensuite, il vérifie si l'utilisateur est connecté et s'il est bien un agriculteur. Si ces conditions ne sont pas respectées, l'utilisateur est redirigé vers la page de connexion. Cette étape permet de sécuriser l'accès à la page et d'empêcher les utilisateurs non autorisés de consulter les offres.

2 : Connexion à la base de données

Le fichier inclut ensuite `database.php`, qui contient la connexion à la base de données via PDO. L'identifiant de l'agriculteur connecté est récupéré depuis la session. Cet identifiant sera utilisé dans toutes les requêtes SQL pour afficher uniquement les données liées à cet utilisateur.

```

$sql = "SELECT o.*, tf.libelle as type_fruit, g.libelle as gouvernorat_libelle,
        (SELECT COUNT(*) FROM uber_cueillette_candidature WHERE id_offre = o.id_offre) as total_candidatures,
        (SELECT COUNT(*) FROM uber_cueillette_candidature WHERE id_offre = o.id_offre AND decision = 'acceptee') as acceptees,
        (SELECT AVG(note) FROM uber_cueillette_candidature WHERE id_offre = o.id_offre AND note IS NOT NULL) as note_moyenne
FROM uber_cueillette_offre o
JOIN uber_cueillette_type_fruit tf ON o.id_type_fruit = tf.id_type_fruit
JOIN uber_cueillette_gouvernorat g ON o.id_gouvernorat = g.id_gouvernorat
WHERE o.idagriculteur = ?
ORDER BY o.date_limite DESC";

$stmt = $pdo->prepare($sql);
$stmt->execute([$idagriculteur]);
$offres = $stmt->fetchAll();

$total_offres = count($offres);
$offres_ouvertes = 0;
$offres_terminees = 0;
$offres_cloturees = 0;
$total_candidatures = 0;

foreach ($offres as $offre) {
    $total_candidatures += $offre['total_candidatures'];

    $aujourd'hui = date('Y-m-d');
    if ($offre['date_fin'] < $aujourd'hui) {
        $offres_terminees++;
    } elseif ($offre['date_limite'] >= $aujourd'hui && $offre['date_debut'] >= $aujourd'hui) {
        $offres_ouvertes++;
    } else {
        $offres_cloturees++;
    }
}
?>

```

3 : Récupération des offres de l'agriculteur

Une requête SQL est exécutée pour récupérer toutes les offres créées par l'agriculteur connecté. Cette requête utilise des jointures avec les tables des types de fruits et des gouvernorats afin d'afficher des informations lisibles (libellés au lieu d'ID).

Elle contient aussi des sous-requêtes pour :

- compter le nombre total de candidatures par offre,
- compter les candidatures acceptées,
- calculer la note moyenne donnée aux ouvriers.

Les résultats sont triés par date limite décroissante afin d'afficher les offres les plus récentes en premier.

Partie 4 : Calcul des statistiques globales

Après la récupération des offres, plusieurs variables sont calculées :

- le nombre total d'offres,
- le nombre d'offres ouvertes,
- le nombre d'offres terminées,
- le nombre d'offres clôturées,
- le nombre total de candidatures.

Une boucle foreach parcourt toutes les offres pour effectuer ces calculs. Cela permet d'avoir une vision globale de l'activité de l'agriculteur.

```

$sql = "SELECT o.*, tf.libelle as type_fruit, g.libelle as gouvernorat_libelle,
        (SELECT COUNT(*) FROM uber_cueillette_candidature WHERE id_offre = o.id_offre) as total_candidatures,
        (SELECT COUNT(*) FROM uber_cueillette_candidature WHERE id_offre = o.id_offre AND decision = 'acceptee') as acceptees,
        (SELECT AVG(note) FROM uber_cueillette_candidature WHERE id_offre = o.id_offre AND note IS NOT NULL) as note_moyenne
FROM uber_cueillette_offre o
JOIN uber_cueillette_type_fruit tf ON o.id_type_fruit = tf.id_type_fruit
JOIN uber_cueillette_gouvernorat g ON o.id_gouvernorat = g.id_gouvernorat
WHERE o.idagriculteur = ?
ORDER BY o.date_limite DESC";

$stmt = $pdo->prepare($sql);
$stmt->execute([$idagriculteur]);
$offres = $stmt->fetchAll();

$total_offres = count($offres);
$offres_ouvertes = 0;
$offres_terminees = 0;
$offres_cloturees = 0;
$total_candidatures = 0;

foreach ($offres as $offre) {
    $total_candidatures += $offre['total_candidatures'];

    $aujourd'hui = date('Y-m-d');
    if ($offre['date_fin'] < $aujourd'hui) {
        $offres_terminees++;
    } elseif ($offre['date_limite'] >= $aujourd'hui && $offre['date_debut'] >= $aujourd'hui) {
        $offres_ouvertes++;
    } else {
        $offres_cloturees++;
    }
}
?>

```

3 : Récupération des offres de l'agriculteur

Une requête SQL est exécutée pour récupérer toutes les offres créées par l'agriculteur connecté. Cette requête utilise des jointures avec les tables des types de fruits et des gouvernorats afin d'afficher des informations lisibles (libellés au lieu d'ID).

Elle contient aussi des sous-requêtes pour :

- compter le nombre total de candidatures par offre,
- compter les candidatures acceptées,
- calculer la note moyenne donnée aux ouvriers.

Les résultats sont triés par date limite décroissante afin d'afficher les offres les plus récentes en premier.

Partie 4 : Calcul des statistiques globales

Après la récupération des offres, plusieurs variables sont calculées :

- le nombre total d'offres,
- le nombre d'offres ouvertes,
- le nombre d'offres terminées,
- le nombre d'offres clôturées,
- le nombre total de candidatures.

Une boucle foreach parcourt toutes les offres pour effectuer ces calculs. Cela permet d'avoir une vision globale de l'activité de l'agriculteur.

5 : Interface de navigation

La barre de navigation permet à l'utilisateur de naviguer entre les différentes pages du système : dashboard, ajout d'offre, gestion des offres et profil. Elle contient également un bouton de déconnexion et un bouton pour changer le thème.

```
<?php if (empty($offres)): ?>
    <div class="empty-state">
        <i class="fas fa-seedling"></i>
        <h3>Aucune offre pour le moment</h3>
        <p>Commencez par créer votre première offre de récolte</p>
        <a href="ajouter-offre.php" class="btn btn-primary">Créer une offre</a>
    </div>
<?php else: ?>
    <?php foreach($offres as $offre):
        $aujourd'hui = date('Y-m-d');

        if ($offre['date_fin'] < $aujourd'hui) {
            $statut = 'terminee';
            $statut_class = 'info';
            $statut_texte = '● Terminée';
        } elseif ($offre['date_limite'] >= $aujourd'hui && $offre['date_debut'] >= $aujourd'hui) {
            $statut = 'ouverte';
            $statut_class = 'success';
            $statut_texte = '● Ouverte';
        } else {
            $statut = 'cloturee';
            $statut_class = 'warning';
            $statut_texte = '● Clôturée';
        }

        $pourcentage = 0;
        if ($offre['nombre_ouvriers'] > 0) {
            $pourcentage = round(($offre['acceptees'] / $offre['nombre_ouvriers']) * 100);
        }
    </foreach>
</?php>
```

6 : Affichage des statistiques

Quatre cartes statistiques affichent les informations principales :

- nombre total d'offres,
- nombre d'offres ouvertes,
- nombre d'offres terminées,
- nombre total de candidatures.

Ces données permettent à l'agriculteur d'avoir un résumé rapide de son activité.

8 : Liste des offres

Si aucune offre n'existe, un message vide est affiché avec un bouton pour créer une nouvelle offre.

Sinon, chaque offre est affichée sous forme de carte dynamique contenant :

type de fruit et gouvernorat, statut (ouverte, clôturée ou terminée), nombre de candidatures, période de récolte, nombre d'ouvriers acceptés, prix journalier, date limite.

Partie 9 : Calcul du statut des offres

Pour chaque offre, le statut est déterminé dynamiquement selon les dates :

Terminée si la date de fin est dépassée,

Ouverte si la période n'a pas encore commencé,

Clôturée sinon.

Cela permet de gérer automatiquement l'état des offres sans intervention manuelle.

Partie 11 : Actions sur les offres

Chaque offre possède plusieurs actions :

voir les postulants,

modifier l'offre (si elle n'est pas terminée),

supprimer l'offre (si aucune candidature n'existe),

noter les ouvriers (si l'offre est terminée),

dupliquer l'offre.

Ces actions permettent une gestion complète des offres.

Partie 10 : Barre de progression

Si une offre est ouverte, une barre de progression est affichée. Elle représente le pourcentage d'ouvriers acceptés par rapport au nombre total demandé. Cela donne une vision claire de l'avancement du recrutement.

Partie 12 : Suppression et duplication

Le JavaScript permet de supprimer une offre via une requête fetch vers le serveur. Après suppression, la carte est retirée dynamiquement du DOM et les statistiques sont mises à jour.

La fonction de duplication redirige vers la page d'ajout d'offre avec les données pré-remplies.

Agriculteur

modifier-offre:

C'est la page qui permet à un agriculteur connecté de modifier une offre de récolte qu'il a déjà publiée.

1 : Sécurisation et session

Le fichier commence par l'ouverture de la session avec `session_start()`. Ensuite, il vérifie si l'utilisateur est connecté et s'il est bien un agriculteur. Si ces conditions ne sont pas respectées, l'utilisateur est redirigé vers la page de connexion. Cette étape permet de sécuriser l'accès à la page et d'empêcher les utilisateurs non autorisés de modifier les offres.

2 : Connexion à la base de données

Le fichier inclut ensuite `database.php`, qui contient la connexion à la base de données via PDO. L'identifiant de l'agriculteur connecté est récupéré depuis la session. Cet identifiant sera utilisé dans toutes les requêtes SQL pour afficher uniquement les données liées à cet utilisateur.

```
// Récupérer les informations de l'offre
$sql = "SELECT o.*, tf.libelle as type_fruit, g.libelle as gouvernorat_libelle
FROM uber_cueillette_offre o
JOIN uber_cueillette_type_fruit tf ON o.id_type_fruit = tf.id_type_fruit
JOIN uber_cueillette_gouvernorat g ON o.id_gouvernorat = g.id_gouvernorat
WHERE o.id_offre = ? AND o.idagriculteur = ?";
$stmt = $pdo->prepare($sql);
$stmt->execute([$id_offre, $idagriculteur]);
$offre = $stmt->fetch();
if (!$offre) {
    header('Location: mes-offres.php');
    exit();
}
// Récupérer les listes pour les selects
$types_fruits = $pdo->query("SELECT * FROM uber_cueillette_type_fruit ORDER BY libelle")->fetchAll();
$gouvernorats = $pdo->query("SELECT * FROM uber_cueillette_gouvernorat ORDER BY libelle")->fetchAll();
$message = '';
$erreur = '';
// Traitement du formulaire
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $type_fruit = $_POST['type_fruit'] ?? '';
    $gouvernorat = $_POST['gouvernorat'] ?? '';
    $adresse = trim($_POST['adresse'] ?? '');
    $date_debut = $_POST['date_debut'] ?? '';
    $date_fin = $_POST['date_fin'] ?? '';
    $date_limite = $_POST['date_limite'] ?? '';
    $nb_ouvriers = $_POST['nb_ouvriers'] ?? '';

    // Récupération du prix avec gestion de l'absence
    if (isset($_POST['prix']) && $_POST['prix'] !== '') {
        $prix = floatval($_POST['prix']);
    } else {
        $prix = $offre['prix_journee'];
    }
}

$erreurs = [];
```

3. Récupération et vérification de l'offre

Le fichier récupère l'identifiant de l'offre dans l'URL et charge ses informations depuis la base de données. Il vérifie que l'offre existe et qu'elle appartient bien à l'agriculteur connecté. Sinon, l'utilisateur est redirigé vers la liste de ses offres.

```
// Validations
if (empty($type_fruit)) $erreurs[] = "Le type de fruit est requis";
if (empty($gouvernorat)) $erreurs[] = "Le gouvernorat est requis";
if (empty($adresse)) $erreurs[] = "L'adresse est requise";
if (empty($date_debut)) $erreurs[] = "La date de début est requise";
if (empty($date_fin)) $erreurs[] = "La date de fin est requise";
if (empty($date_limite)) $erreurs[] = "La date limite est requise";

if (empty($nb_ouvriers) || $nb_ouvriers < 1 || $nb_ouvriers > 50) {
    $erreurs[] = "Le nombre d'ouvriers doit être entre 1 et 50";
}

// Vérification du prix
if ($prix < 10 || $prix > 500) {
    $erreurs[] = "Le prix doit être entre 10 et 500 DT";
}

// Validation des dates
if (!empty($date_debut) && !empty($date_fin) && !empty($date_limite)) {
    $debut = new DateTime($date_debut);
    $fin = new DateTime($date_fin);
    $limite = new DateTime($date_limite);
    $aujourd'hui = new DateTime();
    $aujourd'hui->setTime(0, 0, 0);

    if ($limite < $aujourd'hui && $offre['date_limite'] != $date_limite) {
        $erreurs[] = "La date limite doit être dans le futur";
    }

    if ($fin <= $debut) {
        $erreurs[] = "La date de fin doit être après la date de début";
    }

    if ($limite > $debut) {
        $erreurs[] = "La date limite doit être avant le début de la récolte";
    }
}
```

4. Traitement et validation du formulaire

Lorsque l'agriculteur soumet le formulaire, le fichier récupère les nouvelles valeurs et les valide (champs obligatoires, nombre d'ouvriers entre 1 et 50, prix entre 10 et 500 DT, cohérence des dates). En cas d'erreur, un message s'affiche.

```

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    if (empty($erreurs)) {
        try {
            $sql = "UPDATE uber_cueillette_offre SET
                id_type_fruit = ?,
                id_gouvernorat = ?,
                adresse = ?,
                date_debut = ?,
                date_fin = ?,
                date_limite = ?,
                nombre_ouvriers = ?,
                prix_journee = ?
            WHERE id_offre = ? AND idagriculteur = ?";

            $stmt = $pdo->prepare($sql);
            $result = $stmt->execute([
                $type_fruit,
                $gouvernorat,
                $adresse,
                $date_debut,
                $date_fin,
                $date_limite,
                $nb_ouvriers,
                $prix,
                $id_offre,
                $idagriculteur
            ]);

            if ($result) {
                $message = "✅ Offre modifiée avec succès !";

                // Recharger l'offre
                $stmt = $pdo->prepare("SELECT o.*, tf.libelle as type_fruit, g.libelle as gouvernorat_libelle
                    FROM uber_cueillette_offre o
                    JOIN uber_cueillette_type_fruit tf ON o.id_type_fruit = tf.id_type_fruit
                    JOIN uber_cueillette_gouvernorat g ON o.id_gouvernorat = g.id_gouvernorat
                    WHERE o.id_offre = ?");

                $stmt->execute([$id_offre]);
                $offre = $stmt->fetch();
            }
        }
    }
}

```

```

        if ($result) {
            $message = "✅ Offre modifiée avec succès !";

            // Recharger l'offre
            $stmt = $pdo->prepare("SELECT o.*, tf.libelle as type_fruit, g.libelle as gouvernorat_libelle
                FROM uber_cueillette_offre o
                JOIN uber_cueillette_type_fruit tf ON o.id_type_fruit = tf.id_type_fruit
                JOIN uber_cueillette_gouvernorat g ON o.id_gouvernorat = g.id_gouvernorat
                WHERE o.id_offre = ?");

            $stmt->execute([$id_offre]);
            $offre = $stmt->fetch();
        } else {
            $erreur = "❌ Aucune modification effectuée";
        }
    } catch (PDOException $e) {
        $erreur = "Erreur lors de la modification : " . $e->getMessage();
    }
} else {
    $erreur = implode("<br>", $erreurs);
}

```

5. Mise à jour et affichage du résultat

Si toutes les validations sont réussies, une requête UPDATE modifie l'offre dans la base de données. Un message de succès ou d'erreur est ensuite affiché à l'agriculteur.

Agriculteur

postulants:

Ce fichier permet à l'agriculteur de visualiser tous les ouvriers qui ont postulé à une offre qu'il a publiée, avec leurs profils, leurs notes moyennes et le statut de leur candidature. Il peut ainsi accepter ou refuser chaque candidature directement depuis cette page pour sélectionner les ouvriers qui participeront à sa récolte.

1 : Sécurisation et session

Le fichier commence par l'ouverture de la session avec `session_start()`. Ensuite, il vérifie si l'utilisateur est connecté et s'il est bien un agriculteur. Si ces conditions ne sont pas respectées, l'utilisateur est redirigé vers la page de connexion. Cette étape permet de sécuriser l'accès à la page et d'empêcher les utilisateurs non autorisés de modifier les offres.

2 : Connexion à la base de données

Le fichier inclut ensuite `database.php`, qui contient la connexion à la base de données via PDO. L'identifiant de l'agriculteur connecté est récupéré depuis la session. Cet identifiant sera utilisé dans toutes les requêtes SQL pour afficher uniquement les données liées à cet utilisateur.

```
$sql = "SELECT o.*, tf.libelle as type_fruit, g.libelle as gouvernorat
FROM uber_cueillette_offre o
JOIN uber_cueillette_type_fruit tf ON o.id_type_fruit = tf.id_type_fruit
JOIN uber_cueillette_gouvernorat g ON o.id_gouvernorat = g.id_gouvernorat
WHERE o.id_offre = ? AND o.idagriculteur = ?";
$stmt = $pdo->prepare($sql);
$stmt->execute([$id_offre, $idagriculteur]);
$offre = $stmt->fetch();

if (!$offre) {
    header('Location: mes-offres.php');
    exit();
}

// =====
// PARTIE 4: RÉCUPÉRER TOUS LES POSTULANTS AVEC PHOTO
// =====

$sql = "SELECT c.*, ouv.nom, ouv.prenom, ouv.id_ouvrier, ouv.description, ouv.photo,
(SELECT AVG(note) FROM uber_cueillette_candidature WHERE id_ouvrier = ouv.id_ouvrier AND note IS NOT NULL) as moyenne_notes
FROM uber_cueillette_candidature c
JOIN uber_cueillette_ouvrier ouv ON c.id_ouvrier = ouv.id_ouvrier
WHERE c.id_offre = ?
ORDER BY
CASE c.decision
    WHEN 'en_attente' THEN 1
    WHEN 'acceptee' THEN 2
    ELSE 3
END,
c.date_candidature DESC";
$stmt = $pdo->prepare($sql);
$stmt->execute([$id_offre]);
$postulants = $stmt->fetchAll();
```

Premier SELECT (vérification de l'offre)

Cette requête récupère les informations de l'offre (dates, adresse, nombre d'ouvriers, prix) avec le nom du fruit et du gouvernorat. Elle vérifie que l'offre existe et qu'elle appartient bien à l'agriculteur connecté. Si l'offre n'est pas trouvée, l'utilisateur est redirigé.

Deuxième SELECT (récupération des postulants)

Cette requête récupère tous les ouvriers qui ont postulé à l'offre, avec leurs informations personnelles (nom, prénom, description, photo) et leur moyenne de notes calculée à partir de toutes leurs candidatures précédentes. Les résultats sont triés pour afficher d'abord les candidatures en attente, puis les acceptées, puis les refusées, et par date décroissante dans chaque groupe.

```
// =====
function getPhotoUrl($photo_blob) {
    $default = '../images/default-profile.jpg';
    if (empty($photo_blob)) {
        return $default;
    }
    // Vérifier si c'est un BLOB (données binaires)
    $premier_code = ord(substr($photo_blob, 0, 1));
    // Si le premier caractère n'est pas imprimable ou si la chaîne est très longue
    if ($premier_code < 32 || $premier_code > 126 || strlen($photo_blob) > 255) {
        // C'est un BLOB → encoder en base64
        $photo_data = base64_encode($photo_blob);
        return 'data:image/jpeg;base64,' . $photo_data;
    } else {
        // C'est un chemin de fichier
        $photo_path = '../' . $photo_blob;
        if (file_exists($photo_path)) {
            return $photo_path;
        }
        return $default;
    }
}
// =====
// PARTIE 5: STATISTIQUES DES POSTULANTS
// =====

$total_postulants = count($postulants);
$en_attente = 0;
$acceptes = 0;
$refuses = 0;
foreach ($postulants as $p) {
    if ($p['decision'] == 'en_attente') $en_attente++;
    elseif ($p['decision'] == 'acceptee') $acceptes++;
    elseif ($p['decision'] == 'refusee') $refuses++;
}
$places_restantes = $offre['nombre_ouvriers'] - $acceptes;
$offre_complete = ($acceptes >= $offre['nombre_ouvriers']);
?>
```

Fonction getPhotoUrl()

Cette fonction convertit la photo d'un ouvrier pour l'afficher dans le navigateur. Si la photo est stockée en BLOB (données binaires), elle la transforme en base64. Si c'est un chemin de fichier, elle vérifie que le fichier existe. Si aucune photo n'est trouvée, elle retourne une image par défaut.

Statistiques des postulants

Le code parcourt la liste de tous les postulants pour compter combien sont en attente, acceptés ou refusés. Il calcule ensuite le nombre de places restantes en soustrayant les acceptés du nombre total d'ouvriers demandés dans l'offre, et détermine si l'offre est complète (plus de places disponibles).

JAVASCRIPT (VALIDATION)

FILTRE LES POSTULANTS

LA FONCTION `FILTREPOSTULANTS()` RÉCUPÈRE LA VALEUR DU FILTRE PAR STATUT (TOUS/EN ATTENTE/ACCEPTÉ/REFUSÉ), LE TEXTE DE RECHERCHE (NOM DU POSTULANT) ET LE FILTRE PAR NOTE (3/5/8 ÉTOILES MINIMUM). ELLE PARCOURT TOUTES LES CARTES DES POSTULANTS ET AFFICHE UNIQUEMENT CELLES QUI CORRESPONDENT AUX CRITÈRES SÉLECTIONNÉS. LE COMPTEUR DE RÉSULTATS EST MIS À JOUR EN TEMPS RÉEL ET UN MESSAGE "AUCUN POSTULANT TROUVÉ" APPARAÎT SI NÉCESSAIRE.

RÉINITIALISER LES FILTRES

LA FONCTION `REINITIALISERFILTRES()` REMET LES TROIS FILTRES À LEUR VALEUR PAR DÉFAUT (TOUS LES STATUTS, RECHERCHE VIDE, TOUTES LES NOTES) ET APPELLE `FILTREPOSTULANTS()` POUR RÉAFFICHER TOUS LES POSTULANTS.

ACCEPTER UNE CANDIDATURE

LA FONCTION `ACCEPTERCANDIDATURE(ID)` AFFICHE UNE CONFIRMATION À L'AGRICULTEUR, PUIS ENVOIE UNE REQUÊTE AU FICHIER `REPONDRE-CANDIDATURE.PHP` AVEC L'IDENTIFIANT DE LA CANDIDATURE ET L'ACTION "ACCEPTER". SI LE SERVEUR CONFIRME LE SUCCÈS, LA PAGE SE RECHARGE POUR AFFICHER LA NOUVELLE LISTE MISE À JOUR.

REFUSER UNE CANDIDATURE

LA FONCTION `REFUSERCANDIDATURE(ID)` FONCTIONNE DE LA MÊME MANIÈRE, MAIS AVEC L'ACTION "REFUSER". ELLE ENVOIE LA REQUÊTE AU SERVEUR, AFFICHE UNE CONFIRMATION, PUIS RECHARGE LA PAGE APRÈS VALIDATION.

INITIALISATION AU CHARGEMENT

QUAND LA PAGE EST CHARGÉE, DES ÉCOUTEURS D'ÉVÉNEMENTS SONT ATTACHÉS AUX TROIS FILTRES (CHANGEMENT DE STATUT, SAISIE DANS LA RECHERCHE, CHANGEMENT DE NOTE) ET AUX BOUTONS DE RÉINITIALISATION. LES BOUTONS "ACCEPTER" ET "REFUSER" REÇOIVENT ÉGALEMENT CHACUN UN ÉCOUTEUR DE CLIC QUI APPELLE LA FONCTION CORRESPONDANTE AVEC L'IDENTIFIANT DE LA CANDIDATURE.

Agriculteur

noter-ouvriers:

Cette page permet à l'agriculteur de noter (de 1 à 10 étoiles) les ouvriers qui ont participé à son chantier de récolte, après la fin du travail. Il peut également ajouter un commentaire et indiquer la rémunération versée.

1 : Sécurisation et session

Le fichier commence par l'ouverture de la session avec `session_start()`. Ensuite, il vérifie si l'utilisateur est connecté et s'il est bien un agriculteur. Si ces conditions ne sont pas respectées, l'utilisateur est redirigé vers la page de connexion. Cette étape permet de sécuriser l'accès à la page et d'empêcher les utilisateurs non autorisés de modifier les offres.

2 : Connexion à la base de données

Le fichier inclut ensuite `database.php`, qui contient la connexion à la base de données via PDO. L'identifiant de l'agriculteur connecté est récupéré depuis la session. Cet identifiant sera utilisé dans toutes les requêtes SQL pour afficher uniquement les données liées à cet utilisateur.

```
// Récupérer les ouvriers acceptés pour cette offre
$sql = "SELECT c.*, ouv.nom, ouv.prenom, ouv.id_ouvrier, ouv.photo
        FROM uber_cueillette_candidature c
        JOIN uber_cueillette_ouvrier ouv ON c.id_ouvrier = ouv.id_ouvrier
        WHERE c.id_offre = ? AND c.decision = 'acceptee'";
$stmt = $pdo->prepare($sql);
$stmt->execute([$id_offre]);
$ouvriers = $stmt->fetchAll();

$message = '';
$erreur = '';

// Traitement du formulaire
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $id_candidature = $_POST['id_candidature'] ?? 0;
    $note = $_POST['note'] ?? 0;
    $commentaire = trim($_POST['commentaire'] ?? '');
    $remuneration = $_POST['remuneration'] ?? 0;

    if ($note < 1 || $note > 10) {
        $erreur = "La note doit être entre 1 et 10";
    } else {
        $sql = "UPDATE uber_cueillette_candidature SET note = ?, commentaire = ?, remuneration = ? WHERE id_candidature = ?";
        $stmt = $pdo->prepare($sql);
        if ($stmt->execute([$note, $commentaire, $remuneration, $id_candidature])) {
            $message = "✅ Notation enregistrée !";
        } else {
            $erreur = "❌ Erreur lors de l'enregistrement";
        }
    }
}

// Recharger les données après mise à jour
$stmt = $pdo->prepare("SELECT c.*, ouv.nom, ouv.prenom, ouv.id_ouvrier, ouv.photo
                      FROM uber_cueillette_candidature c
                      JOIN uber_cueillette_ouvrier ouv ON c.id_ouvrier = ouv.id_ouvrier
                      WHERE c.id_offre = ? AND c.decision = 'acceptee'
                      ORDER BY c.note IS NULL DESC, c.note DESC");
$stmt->execute([$id_offre]);
$ouvriers = $stmt->fetchAll();
```

Premier SELECT : Vérifie que l'offre existe et appartient bien à l'agriculteur connecté. Il récupère les informations de l'offre (fruit, gouvernorat, dates, prix, nombre d'ouvriers).

Deuxième SELECT : Récupère tous les ouvriers qui ont été acceptés pour cette offre (ceux qui ont participé au chantier), avec leurs noms, prénoms et photos. Les ouvriers déjà notés apparaissent en bas de liste grâce à ORDER BY c.note IS NULL DESC.

```
// ✅ FONCTION CORRIGÉE POUR AFFICHER LA PHOTO (GESTION NULL)
function getPhoto($ouvrier) {
    $default = '../images/default-profile.jpg';

    // ✅ CORRECTION 1: Vérifier si la photo est vide ou NULL
    if (empty($ouvrier['photo'])) {
        return $default;
    }

    // Vérifier si c'est un BLOB
    $premier_code = ord(substr($ouvrier['photo'], 0, 1));
    if ($premier_code < 32 || $premier_code > 126 || strlen($ouvrier['photo']) > 255) {
        $photo_data = base64_encode($ouvrier['photo']);
        return 'data:image/jpeg;base64,' . $photo_data;
    } else {
        $photo_path = '../' . $ouvrier['photo'];
        if (file_exists($photo_path)) {
            return $photo_path;
        }
    }
    return $default;
}
```

Fonction getPhoto() : Gère l'affichage des photos des ouvriers. Si la photo est vide ou NULL, elle retourne une image par défaut. Si c'est un BLOB, elle le convertit en base64. Si c'est un chemin, elle vérifie que le fichier existe.

JAVASCRIPT (VALIDATION)

SYSTÈME D'ÉTOILES INTERACTIF

LA FONCTION PARCOURT TOUS LES CONTENEURS D'ÉTOILES ET ATTACHE DES ÉVÉNEMENTS À CHAQUE ÉTOILE. AU SURVOL, LES ÉTOILES S'ALLUMENT EN SURBRILLANCE JAUNE. AU CLIC, LA NOTE EST FIGÉE ET STOCKÉE DANS UN CHAMP CACHÉ. UN TOAST DE CONFIRMATION APPARAÎT POUR INDiquer LA NOTE SÉLECTIONNÉE.

AFFICHAGE DU TOAST

LA FONCTION SHOWTOAST() CRÉE UNE PETITE NOTIFICATION QUI APPARAÎT EN BAS À DROITE DE L'ÉCRAN LORSQUE L'AGRICULTEUR SÉLECTIONNE UNE NOTE. LE MESSAGE S'AFFICHE PENDANT 2 SECONDES AVEC UNE ANIMATION DE DISPARITION PROGRESSIVE.

VALIDATION AVANT SOUMISSION

LES FORMULAIRES DE NOTATION SONT TOUS VÉRIFIÉS AVANT ENVOI. SI AUCUNE NOTE N'A ÉTÉ SÉLECTIONNÉE (OU SI LA NOTE N'EST PAS ENTRE 1 ET 10), L'ENVOI EST BLOQUÉ ET UNE ALERTE S'AFFICHE. UNE CONFIRMATION SUPPLÉMENTAIRE DEMANDE À L'AGRICULTEUR DE VALIDER SON CHOIX AVANT L'ENVOI DÉFINITIF.

INITIALISATION AU CHARGEMENT

QUAND LA PAGE EST CHARGÉE, LES ÉVÉNEMENTS SONT ATTACHÉS À TOUTES LES ÉTOILES DE CHAQUE FORMULAIRE. POUR LES OUVRIERS DÉJÀ NOTÉS, LE SYSTÈME AFFICHE SIMPLEMENT LES ÉTOILES CORRESPONDANT À LEUR NOTE SANS FORMULAIRE INTERACTIF. POUR CEUX NON NOTÉS, LE FORMULAIRE COMPLET AVEC LES ÉTOILES CLIQUABLES EST ACTIVÉ.

Agriculteur

profil-ouvrier:

Cette page permet à l'agriculteur de consulter le profil détaillé d'un ouvrier qui a postulé à ses offres. Il peut voir sa photo, ses informations personnelles, sa note moyenne, ses commentaires reçus et l'historique de ses chantiers.

1 : Sécurisation et session

Le fichier commence par l'ouverture de la session avec `session_start()`. Ensuite, il vérifie si l'utilisateur est connecté et s'il est bien un agriculteur. Si ces conditions ne sont pas respectées, l'utilisateur est redirigé vers la page de connexion. Cette étape permet de sécuriser l'accès à la page et d'empêcher les utilisateurs non autorisés de modifier les offres.

2 : Connexion à la base de données

Le fichier inclut ensuite `database.php`, qui contient la connexion à la base de données via PDO. L'identifiant de l'agriculteur connecté est récupéré depuis la session. Cet identifiant sera utilisé dans toutes les requêtes SQL pour afficher uniquement les données liées à cet utilisateur.

```
$sql = "SELECT * FROM uber_cueillette_ouvrier WHERE id_ouvrier = ?";
$stmt = $pdo->prepare($sql);
$stmt->execute([$id_ouvrier]);
$ouvrier = $stmt->fetch();

if (!$ouvrier) {
    header('Location: mes-offres.php');
    exit();
}

// =====
// PARTIE 4: RÉCUPÉRER LES STATISTIQUES DE L'OUVRIER
// =====
// Note moyenne
$sql = "SELECT AVG(note) as moyenne, COUNT(*) as nb_notes
        FROM uber_cueillette_candidature
        WHERE id_ouvrier = ? AND note IS NOT NULL";
$stmt = $pdo->prepare($sql);
$stmt->execute([$id_ouvrier]);
$stats = $stmt->fetch();
$moyenne = $stats['moyenne'] ? number_format($stats['moyenne'], 1) : 'N/A';
$nb_notes = $stats['nb_notes'] ?? 0;
// Nombre de chantiers effectués
$sql = "SELECT COUNT(*) as total
        FROM uber_cueillette_candidature
        WHERE id_ouvrier = ? AND decision = 'acceptee'";
$stmt = $pdo->prepare($sql);
$stmt->execute([$id_ouvrier]);
$nb_chantiers = $stmt->fetch()['total'] ?? 0;
// Derniers commentaires reçus
$sql = "SELECT c.*, o.adresse, tf.libelle as type_fruit,
        o.date_debut, o.date_fin, o.prix_journee
        FROM uber_cueillette_candidature c
        JOIN uber_cueillette_offre o ON c.id_offre = o.id_offre
        JOIN uber_cueillette_type_fruit tf ON o.id_type_fruit = tf.id_type_fruit
        WHERE c.id_ouvrier = ? AND c.commentaire IS NOT NULL
        ORDER BY c.date_candidature DESC
        LIMIT 5";
$stmt = $pdo->prepare($sql);
```

Premier SELECT : Récupère toutes les informations personnelles de l'ouvrier (nom, prénom, CIN, email, description, photo) à partir de son identifiant.

Deuxième SELECT : Calcule la note moyenne de l'ouvrier et le nombre total d'avis reçus à partir de la table des candidatures.

Troisième SELECT : Compte le nombre de chantiers que l'ouvrier a effectués (candidatures acceptées).

Quatrième SELECT : Récupère les 5 derniers commentaires reçus par l'ouvrier, avec les détails des offres (fruit, adresse, dates, prix).

```
$sql = "SELECT o.*, tf.libelle as type_fruit, g.libelle as gouvernorat,
        c.note, c.commentaire, c.remuneration, c.decision
        FROM uber_cueillette_candidature c
        JOIN uber_cueillette_offre o ON c.id_offre = o.id_offre
        JOIN uber_cueillette_type_fruit tf ON o.id_type_fruit = tf.id_type_fruit
        JOIN uber_cueillette_gouvernorat g ON o.id_gouvernorat = g.id_gouvernorat
        WHERE c.id_ouvrier = ? AND c.decision = 'acceptee'
        ORDER BY o.date_fin DESC
        LIMIT 3";
$stmt = $pdo->prepare($sql);
$stmt->execute([$id_ouvrier]);
$chantiers = $stmt->fetchAll();

// =====
// PARTIE 5: PHOTO DE PROFIL (CORRECTION BLOB)
// =====
$photo_url = '../images/default-profile.jpg'; // Photo par défaut
if (!empty($ouvrier['photo'])) {
    // Vérifier si c'est un BLOB (données binaires)
    // Méthode: vérifier si la chaîne contient des caractères non imprimables
    $is_blob = false;
    $premier_caractere = substr($ouvrier['photo'], 0, 1);
    $premier_code = ord($premier_caractere);
    // Si le premier caractère n'est pas imprimable ou si la chaîne est très longue
    if ($premier_code < 32 || $premier_code > 126 || strlen($ouvrier['photo']) > 255) {
        $is_blob = true;
    }
    if ($is_blob) {
        // C'est un BLOB → encoder en base64 pour l'affichage
        $photo_data = base64_encode($ouvrier['photo']);
        $photo_url = 'data:image/jpeg;base64,' . $photo_data;
    } else {
        // C'est un chemin de fichier
        $photo_path = '../' . $ouvrier['photo'];
        if (file_exists($photo_path)) {
            $photo_url = $photo_path;
        }
    }
}
```

Cinquième SELECT : Récupère l'historique des 3 derniers chantiers effectués par l'ouvrier, avec les notes et rémunérations associées.

Traitement de la photo

Le code vérifie si la photo est un BLOB (données binaires) ou un chemin de fichier. Si c'est un BLOB (premier caractère non imprimable ou chaîne trop longue), il la convertit en base64 pour l'affichage. Si c'est un chemin, il vérifie que le fichier existe. Si aucune photo n'est trouvée, une image par défaut est utilisée.

Affichage des statistiques

La page affiche quatre statistiques sous forme de cartes : la note moyenne de l'ouvrier, le nombre de chantiers effectués, le nombre de commentaires reçus, et la date d'inscription (membre depuis)

JAVASCRIPT (VALIDATION)

ANIMATION DES STATISTIQUES : LA FONCTION ANIMERSTATISTIQUES() PARCOURT TOUS LES NOMBRES AFFICHÉS DANS LES STATISTIQUES ET LES FAIT AUGMENTER PROGRESSIVEMENT DE 0 JUSQU'À LEUR VALEUR FINALE, AVEC UN INCRÉMENT DE 0,1 TOUTES LES 50 MILLISECONDES. LES DATES ET LES VALEURS "N/A" NE SONT PAS ANIMÉES.

AGRANDISSEMENT DE LA PHOTO : LA FONCTION INITPHOTOZOOM() ATTACHE UN ÉCOUTEUR DE CLIC SUR LA PHOTO DE PROFIL. QUAND L'AGRICULTEUR CLIQUE DESSUS, UNE MODALE NOIRE S'OUVRE AVEC LA PHOTO AGRANDIE AU CENTRE DE L'ÉCRAN. UN SECOND CLIC FERME LA MODALE.

INITIALISATION AU CHARGEMENT : QUAND LA PAGE EST CHARGÉE, LES ANIMATIONS DES STATISTIQUES SE LANCENT AUTOMATIQUEMENT, LA FONCTION D'AGRANDISSEMENT DE LA PHOTO EST ACTIVÉE, ET DES TOOLTIPS SONT AJOUTÉS AUX ÉTOILES DE NOTATION POUR INDiquer "NOTE SUR 10". UN GESTIONNAIRE D'ÉVÉNEMENT RECHARGE LA PAGE SI L'UTILISATEUR REVIENT EN ARRIÈRE AVEC LA TOUCHE DU NAVIGATEUR.

Agriculteur

profil:

Cette page permet à l'agriculteur de consulter et modifier ses informations personnelles (nom, prénom, email, adresse) ainsi que de changer son mot de passe. Les champs CIN et pseudo sont non modifiables pour des raisons de sécurité.

1 : Sécurisation et session

Le fichier commence par l'ouverture de la session avec `session_start()`. Ensuite, il vérifie si l'utilisateur est connecté et s'il est bien un agriculteur. Si ces conditions ne sont pas respectées, l'utilisateur est redirigé vers la page de connexion. Cette étape permet de sécuriser l'accès à la page et d'empêcher les utilisateurs non autorisés de modifier les offres.

2 : Connexion à la base de données

Le fichier inclut ensuite `database.php`, qui contient la connexion à la base de données via PDO. L'identifiant de l'agriculteur connecté est récupéré depuis la session. Cet identifiant sera utilisé dans toutes les requêtes SQL pour afficher uniquement les données liées à cet utilisateur.

```
// Récupérer les informations actuelles
$sql = "SELECT * FROM uber_cueilletteagriculteur WHERE idagriculteur = ?";
$stmt = $pdo->prepare($sql);
$stmt->execute([$idagriculteur]);
$agriculteur = $stmt->fetch();
$message = '';
$erreur = '';
// Traitement du formulaire de modification
if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_POST['modifier'])) {
    $nom = trim($_POST['nom'] ?? '');
    $prenom = trim($_POST['prenom'] ?? '');
    $email = trim($_POST['email'] ?? '');
    $adresse = trim($_POST['adresse'] ?? '');
    $telephone = trim($_POST['telephone'] ?? '');

    $erreurs = [];

    if (empty($nom)) $erreurs[] = "Le nom est requis";
    if (empty($prenom)) $erreurs[] = "Le prénom est requis";
    if (empty($email)) $erreurs[] = "L'email est requis";
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) $erreurs[] = "Email invalide";

    if (empty($erreurs)) {
        $sql = "UPDATE uber_cueilletteagriculteur SET nom = ?, prenom = ?, email = ?, adresse = ? WHERE idagriculteur = ?";
        $stmt = $pdo->prepare($sql);
        if ($stmt->execute([$nom, $prenom, $email, $adresse, $idagriculteur])) {
            $message = "✅ Profil mis à jour avec succès !";
            // Recharger les données
            $stmt = $pdo->prepare("SELECT * FROM uber_cueilletteagriculteur WHERE idagriculteur = ?");
            $stmt->execute([$idagriculteur]);
            $agriculteur = $stmt->fetch();
        } else {
            $erreur = "❌ Erreur lors de la mise à jour";
        }
    } else {
        $erreur = implode("<br>", $erreurs);
    }
}
```

Récupération des informations

Un premier SELECT récupère toutes les informations de l'agriculteur connecté à partir de son identifiant stocké en session. Ces données sont affichées dans le formulaire.

Modification des informations personnelles

Quand l'agriculteur soumet le formulaire de modification, le code récupère le nom, prénom, email et adresse. Il valide que les champs ne sont pas vides et que l'email est valide. Si tout est correct, une requête UPDATE met à jour la base de données et affiche un message de succès. Les données du profil sont rechargées pour afficher les nouvelles valeurs.

```
// Traitement du changement de mot de passe
if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_POST['changer_mdp'])) {
    $ancien = $_POST['ancien_mdp'] ?? '';
    $nouveau = $_POST['nouveau_mdp'] ?? '';
    $confirmation = $_POST['confirmation_mdp'] ?? '';

    $erreurs = [];

    if (empty($ancien)) $erreurs[] = "L'ancien mot de passe est requis";
    if (empty($nouveau)) $erreurs[] = "Le nouveau mot de passe est requis";
    if ($nouveau != $confirmation) $erreurs[] = "Les mots de passe ne correspondent pas";
    if (strlen($nouveau) < 8) $erreurs[] = "Le mot de passe doit contenir au moins 8 caractères";
    if (!preg_match('/[$#]$/ ', $nouveau)) $erreurs[] = "Le mot de passe doit se terminer par $ ou #";

    if (empty($erreurs)) {
        if ($ancien == $agriculteur['password']) {
            $sql = "UPDATE uber_cueilletteagriculteur SET password = ? WHERE idagriculteur = ?";
            $stmt = $pdo->prepare($sql);
            if ($stmt->execute([$nouveau, $idagriculteur])) {
                $message = "✅ Mot de passe changé avec succès !";
            } else {
                $erreur = "❌ Erreur lors du changement";
            }
        } else {
            $erreur = "❌ Ancien mot de passe incorrect";
        }
    } else {
        $erreur = implode("<br>", $erreurs);
    }
}
```

Changement du mot de passe

Quand l'agriculteur soumet le formulaire de changement de mot de passe, le code récupère l'ancien mot de passe, le nouveau et sa confirmation. Il valide que le nouveau mot de passe fait au moins 8 caractères et se termine par \$ ou #, et que les deux mots de passe correspondent. Si l'ancien mot de passe est correct, la base de données est mise à jour avec le nouveau mot de passe.

Affichage des messages

La page affiche en haut un message de succès (vert) si l'opération a réussi, ou un message d'erreur (rouge) si une validation a échoué (champ vide, email invalide, mot de passe incorrect, etc.).

Champs non modifiables

Le CIN et le pseudo sont affichés dans des champs en lecture seule (readonly disabled) car ces informations ne doivent pas être modifiées après l'inscription.

Traitement

ajouter_offre:

Ce fichier sert à traiter l'ajout d'une nouvelle offre de récolte par un agriculteur. Il reçoit les données envoyées depuis un formulaire (page ajouter-offre), les vérifie, puis les enregistre dans la base de données si tout est correct. Il assure aussi la sécurité en vérifiant que l'utilisateur est bien connecté en tant qu'agriculteur.

1.Vérification de la session

Au début, le système démarre une session et vérifie si l'utilisateur est connecté et s'il est bien de type agriculteur. Si ce n'est pas le cas, il est redirigé vers la page de connexion avec un message d'erreur. Cela permet d'éviter l'accès non autorisé au traitement.

```
// Récupération des données
$type_fruit = $_POST['type_fruit'] ?? '';
$gouvernorat = $_POST['gouvernorat'] ?? '';
$adresse = trim($_POST['adresse'] ?? '');
$date_debut = $_POST['date_debut'] ?? '';
$date_fin = $_POST['date_fin'] ?? '';
$date_limite = $_POST['date_limite'] ?? '';
$nb_ouvriers = $_POST['nb_ouvriers'] ?? '';
$prix = $_POST['prix'] ?? '';
$description = trim($_POST['description'] ?? '');
$conditions = isset($_POST['conditions']);

$erreurs = [];
```

2.Récupération des données

Le script récupère ensuite les données envoyées par le formulaire via POST, comme le type de fruit, le gouvernorat, l'adresse, les dates, le nombre d'ouvriers, le prix et la description. Ces informations représentent les détails de l'offre à créer.

```
// Validations
if (empty($type_fruit)) $erreurs[] = "Le type de fruit est requis";
if (empty($gouvernorat)) $erreurs[] = "Le gouvernorat est requis";
if (empty($adresse)) $erreurs[] = "L'adresse est requise";
if (strlen($adresse) < 5) $erreurs[] = "L'adresse doit contenir au moins 5 caractères";
if (empty($date_debut)) $erreurs[] = "La date de début est requise";
if (empty($date_fin)) $erreurs[] = "La date de fin est requise";
if (empty($date_limite)) $erreurs[] = "La date limite est requise";

if (!empty($date_debut) && !empty($date_fin) && !empty($date_limite)) {
    $debut = new DateTime($date_debut);
    $fin = new DateTime($date_fin);
    $limite = new DateTime($date_limite);
    $aujourd'hui = new DateTime();
    $aujourd'hui->setTime(0, 0, 0);

    if ($limite < $aujourd'hui) $erreurs[] = "La date limite doit être dans le futur";
    if ($fin <= $debut) $erreurs[] = "La date de fin doit être après la date de début";
}

if (empty($nb_ouvriers) || $nb_ouvriers < 1 || $nb_ouvriers > 50) $erreurs[] = "Le nombre d'ouvriers doit être entre 1 et 50";
if (empty($prix) || $prix < 10 || $prix > 500) $erreurs[] = "Le prix doit être entre 10 et 500 DT";
if (!$conditions) $erreurs[] = "Vous devez accepter les conditions générales";

if (!empty($erreurs)) {
    $_SESSION['erreurs'] = $erreurs;
    $_SESSION['old'] = $_POST;
    header('Location: ../agriculteur/ajouter-offre.php');
    exit();
}
```

3.Validation des données

Une étape importante consiste à vérifier la validité des données. Le code contrôle si les champs sont remplis, si les dates sont cohérentes (date de fin après la date de début, date limite dans le futur), et si les valeurs comme le prix ou le nombre d'ouvriers sont dans des limites acceptables. Si une erreur est détectée, elle est stockée dans une liste.

4. Gestion des erreurs

Si des erreurs existent, elles sont enregistrées dans la session (\$_SESSION['erreurs']) avec les anciennes données du formulaire (\$_SESSION['old']). Ensuite, l'utilisateur est redirigé vers la page ajouter-offre.php pour corriger les erreurs sans perdre les informations déjà saisies.

```
try {
    $sql = "INSERT INTO uber_cueillette_offre
        (idagriculteur, id_type_fruit, id_gouvernorat, adresse, date_debut, date_fin, date_limite, nombre_ouvriers, prix_journee, description)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";

    $stmt = $pdo->prepare($sql);
    $stmt->execute([
        $idagriculteur,
        $type_fruit,
        $gouvernorat,
        $adresse,
        $date_debut,
        $date_fin,
        $date_limite,
        $nb_ouvriers,
        $prix,
        $description // ✅ CORRECTION : ajout de la description
    ]);

    $_SESSION['success'] = "✅ Offre ajoutée avec succès !";
    header('Location: ../agriculteur/mes-offres.php');
    exit();
} catch (PDOException $e) {
    $_SESSION['erreurs'] = ["Erreur lors de l'ajout : " . $e->getMessage()];
    $_SESSION['old'] = $_POST;
    header('Location: ../agriculteur/ajouter-offre.php');
    exit();
}
?>
```

5. Insertion dans la base de données

Si aucune erreur n'est détectée, le script insère la nouvelle offre dans la base de données avec une requête SQL INSERT. Les informations sont stockées dans la table uber_cueillette_offre en utilisant une requête préparée pour éviter les injections SQL.

6. Redirection après succès ou échec

Si l'insertion réussit, un message de succès est stocké dans la session et l'utilisateur est redirigé vers la page mes-offres.php. En cas d'erreur technique (ex: problème base de données), un message d'erreur est affiché et l'utilisateur retourne au formulaire.

Traitement

repondre_condidature:

Ce fichier sert à traiter la réponse de l'agriculteur à une candidature. Il permet soit d'accepter, soit de refuser un ouvrier qui a postulé à une offre.

1.Vérification de la session

Au début, le script vérifie que l'utilisateur est bien connecté et qu'il est de type agriculteur. Sinon, il bloque l'accès et renvoie un message JSON "Non autorisé". Cela sécurise le traitement pour éviter toute utilisation frauduleuse.

2.Connexion à la base de données

Le fichier inclut ensuite la connexion à la base de données via database.php, afin de pouvoir lire et modifier les informations des candidatures dans les tables.

3. Récupération et validation des données

Le script récupère les données envoyées en POST : l'ID de la candidature et l'action (accepter ou refuser). Il vérifie ensuite que ces données sont valides. Si les paramètres sont incorrects, une réponse JSON d'erreur est envoyée.

```
// =====  
// 3. RÉCUPÉRER LES DONNÉES (uniquement POST)  
// =====  
  
$id_candidature = isset($_POST['id']) ? intval($_POST['id']) : 0;  
$action = isset($_POST['action']) ? $_POST['action'] : '';  
  
if ($id_candidature <= 0 || !in_array($action, ['accepter', 'refuser'])) {  
    echo json_encode(['success' => false, 'message' => 'Paramètres invalides']);  
    exit();  
}
```

4. Détermination de la décision

Selon l'action reçue, le système transforme la valeur en une décision interne :

“accepter” devient acceptee

“refuser” devient refusee

Cela permet de standardiser les valeurs stockées dans la base de données.

```
// =====  
// 4. DÉTERMINER LA NOUVELLE DÉCISION  
// =====  
  
$decision = ($action == 'accepter') ? 'acceptee' : 'refusee';
```

5.Vérification de la candidature

Le script vérifie ensuite que la candidature existe bien et qu'elle appartient à l'agriculteur connecté. Il récupère aussi le nombre d'ouvriers déjà acceptés pour cette offre afin d'éviter les dépassements.

```
// =====  
// 5. VÉRIFIER QUE LA CANDIDATURE APPARTIENT BIEN À L'AGRICULTEUR  
// =====  
  
$sql = "SELECT c.*, o.id_agriculteur, o.nombre_ouvriers,  
        (SELECT COUNT(*) FROM uber_cueillette_candidature  
         WHERE id_offre = o.id_offre AND decision = 'acceptee') as nb_acceptes  
        FROM uber_cueillette_candidature c  
        JOIN uber_cueillette_offre o ON c.id_offre = o.id_offre  
        WHERE c.id_candidature = ?";  
$stmt = $pdo->prepare($sql);  
$stmt->execute([$id_candidature]);  
$candidature = $stmt->fetch();  
  
if (!$candidature) {  
    echo json_encode(['success' => false, 'message' => 'Candidature introuvable']);  
    exit();  
}  
  
if ($candidature['id_agriculteur'] != $_SESSION['user_id']) {  
    echo json_encode(['success' => false, 'message' => 'Non autorisé']);  
    exit();  
}
```

6.Vérification des places disponibles

Si l'agriculteur essaie d'accepter une candidature alors que le nombre maximum d'ouvriers est atteint, le système bloque l'action et renvoie un message indiquant que l'offre est complète.

```
// =====  
// 6. VÉRIFIER LES PLACES DISPONIBLES (SI ON ACCEPTE)  
// =====  
  
if ($action == 'accepter' && $candidature['nb_acceptes'] >= $candidature['nombre_ouvriers']) {  
    echo json_encode(['success' => false, 'message' => 'Offre complète, plus de places disponibles']);  
    exit();  
}
```

7. Mise à jour dans la base de données

Si tout est correct, la décision est enregistrée dans la base de données en mettant à jour le champ decision de la candidature. Ensuite, une réponse JSON de succès est envoyée.

8. Gestion des erreurs

Si une erreur survient lors de la mise à jour (problème SQL ou autre), le script capture l'erreur et renvoie un message JSON avec le détail du problème.

```
// =====  
// 7. METTRE À JOUR LA BASE DE DONNÉES  
// =====  
  
try {  
    $sql = "UPDATE uber_cueillette_candidature SET decision = ? WHERE id_candidature = ?";  
    $stmt = $pdo->prepare($sql);  
    $result = $stmt->execute([$decision, $id_candidature]);  
  
    if ($result) {  
        echo json_encode(['success' => true]);  
    } else {  
        echo json_encode(['success' => false, 'message' => 'Erreur lors de la mise à jour']);  
    }  
} catch (Exception $e) {  
    echo json_encode(['success' => false, 'message' => $e->getMessage()]);  
}  
?>
```

Traitement

supprimer_offre:

Ce fichier permet de supprimer une offre de récolte créée par un agriculteur. Cependant, la suppression n'est possible que si certaines conditions sont respectées, notamment l'absence de candidatures liées à cette offre.

1.Vérification de la session

Le script commence par vérifier que l'utilisateur est bien connecté et qu'il est de type agriculteur. Si ce n'est pas le cas, il bloque l'accès et renvoie une réponse au format JSON indiquant que l'utilisateur n'est pas autorisé.

2.Connexion à la base de données

Ensuite, le fichier inclut la connexion à la base de données afin de pouvoir accéder aux informations des offres et des candidatures.

3. Récupération de l'ID de l'offre

Le script récupère l'identifiant de l'offre à supprimer depuis l'URL (GET). Si cet identifiant est manquant ou vide, une erreur est renvoyée pour empêcher toute action incorrecte.

4. Vérification de la propriété de l'offre

Avant de supprimer l'offre, le système vérifie qu'elle existe bien et qu'elle appartient à l'agriculteur connecté. Cela empêche un utilisateur de supprimer une offre qui ne lui appartient pas.

```
// =====  
// PARTIE 4: VÉRIFIER QUE L'OFFRE APPARTIENT À L'AGRICULTEUR  
// =====  
  
$sql = "SELECT id_offre, id_agriculteur FROM uber_cueillette_offre WHERE id_offre = ?";  
$stmt = $pdo->prepare($sql);  
$stmt->execute([$id_offre]);  
$offre = $stmt->fetch();  
  
if (!$offre) {  
    header('Content-Type: application/json');  
    echo json_encode(['success' => false, 'message' => 'Offre introuvable']);  
    exit();  
}  
  
if ($offre['id_agriculteur'] != $id_agriculteur) {  
    header('Content-Type: application/json');  
    echo json_encode(['success' => false, 'message' => 'Vous n\'êtes pas autorisé à supprimer cette offre']);  
    exit();  
}
```

5.Vérification des candidatures

Une étape essentielle consiste à vérifier si l'offre possède déjà des candidatures. Si au moins une candidature existe, la suppression est refusée. Cela permet de protéger les données et d'éviter la perte d'informations importantes.

```
// =====  
// PARTIE 5: VÉRIFIER QU'IL N'Y A PAS DE CANDIDATURES  
// =====  
  
$sql = "SELECT COUNT(*) as total FROM uber_cueillette_candidature WHERE id_offre = ?";  
$stmt = $pdo->prepare($sql);  
$stmt->execute([$id_offre]);  
$resultat = $stmt->fetch();  
$nb_candidatures = $resultat['total'] ?? 0;  
  
if ($nb_candidatures > 0) {  
    header('Content-Type: application/json');  
    echo json_encode(['success' => false, 'message' => 'Impossible de supprimer : cette offre a ' . $nb_candidatures . ' candidature(s)']);  
    exit();  
}
```

6.Suppression de l'offre

Si toutes les conditions sont respectées, l'offre est supprimée de la base de données grâce à une requête SQL DELETE. Une réponse JSON de succès est ensuite envoyée pour informer que l'opération a réussi.

```
// =====  
// PARTIE 6: SUPPRIMER L'OFFRE  
// =====  
  
try {  
    $sql = "DELETE FROM uber_cueillette_offre WHERE id_offre = ?";  
    $stmt = $pdo->prepare($sql);  
    $stmt->execute([$id_offre]);  
  
    header('Content-Type: application/json');  
    echo json_encode(['success' => true, 'message' => 'Offre supprimée avec succès']);  
} catch (PDOException $e) {  
    header('Content-Type: application/json');  
    echo json_encode(['success' => false, 'message' => 'Erreur lors de la suppression : ' . $e->getMessage()]);  
}  
?>
```

Ouvrier

dashboard:

C'est la page d'accueil de l'ouvrier après sa connexion. Elle affiche un résumé de son activité sur la plateforme (chantiers réalisés, gains, note moyenne, candidatures en attente) ainsi que les offres de récolte disponibles auxquelles il peut postuler.

1. Vérification de l'accès

Le PHP démarre la session et vérifie que l'utilisateur est connecté et qu'il est bien un ouvrier. Si ce n'est pas le cas, il est redirigé vers la page de connexion.

2. Récupération des informations de l'ouvrier

Le PHP exécute une requête SELECT pour récupérer le nom, prénom, pseudo et email de l'ouvrier connecté à partir de son ID stocké en session.

3. Calcul des statistiques personnelles

Le PHP exécute plusieurs requêtes pour calculer : le nombre de chantiers réalisés (candidatures acceptées), le total des gains cumulés, la note moyenne reçue, et le nombre de candidatures encore en attente de réponse.

4. Recherche des offres disponibles

Le PHP récupère les 5 offres actives (date limite et date fin dans le futur, places encore libres), avec le nom du fruit et du gouvernorat, triées par date limite croissante (offres les plus urgentes en premier).

5. Affichage des données dans la page

Toutes ces données (statistiques, offres disponibles) sont injectées dans le HTML pour être affichées à l'ouvrier sous forme de cartes et de chiffres.

Ouvrier

mes-candidatures:

Cette page permet à l'ouvrier de consulter toutes ses candidatures envoyées aux agriculteurs, avec leur statut (en attente, acceptée ou refusée) et les détails de chaque offre.

1. Démarrage de la session et vérification de l'accès

Le PHP démarre la session avec `session_start()`. Il vérifie que l'utilisateur est connecté et qu'il est bien un ouvrier. Si ce n'est pas le cas, l'utilisateur est redirigé vers la page de connexion.

2. Connexion à la base de données

Le PHP inclut le fichier `database.php` pour pouvoir interroger la base de données. Il récupère l'identifiant de l'ouvrier stocké dans la session.

3. Récupération de toutes les candidatures de l'ouvrier

Le PHP exécute une requête `SELECT` avec plusieurs `JOIN` :

- `uber_cueillette_candidature` pour les candidatures
- `uber_cueillette_offre` pour les offres
- `uber_cueillette_type_fruit` pour le nom du fruit
- `uber_cueillette_gouvernorat` pour le nom du gouvernorat

La requête récupère le statut de chaque candidature, l'adresse, les dates, le prix, et trie les résultats par date de candidature décroissante (les plus récentes d'abord).

4. Calcul des statistiques des candidatures

Le PHP parcourt en boucle toutes les candidatures récupérées. Pour chacune, il vérifie la valeur du champ `decision` (`en_attente`, `acceptee`, `refusee`) et incrémente le compteur correspondant. Ces statistiques sont ensuite affichées en haut de page.

5. Affichage conditionnel

Le PHP vérifie si le tableau de candidatures est vide ou non. S'il est vide, un message "Aucune candidature" s'affiche avec un bouton pour explorer les offres. S'il contient des candidatures, le PHP génère le HTML pour afficher chaque candidature sous forme de carte avec son statut et ses informations.

6. Attribution des classes CSS selon le statut

Pour chaque candidature, le PHP détermine la classe CSS et le texte à afficher en fonction du statut : `"badge-warning"` avec "En attente" pour les candidatures en attente, `"badge-success"` avec "Acceptée" pour les acceptées, `"badge-danger"` avec "Refusée" pour les refusées.

Ouvrier

JAVASCRIPT (VALIDATION)

Filtrer les candidatures

La fonction `filtrerCandidatures()` récupère la valeur du filtre par statut et le texte de recherche. Elle parcourt toutes les cartes des candidatures et affiche uniquement celles dont le statut correspond au filtre et dont le fruit ou le gouvernorat contient le texte recherché. Le compteur de résultats est mis à jour en temps réel.

Réinitialiser les filtres

La fonction `reinitialiserFiltres()` remet le filtre par statut à "Tous" et vide le champ de recherche. Elle appelle ensuite `filtrerCandidatures()` pour réafficher toutes les candidatures.

Initialisation au chargement

Quand la page est chargée, des écouteurs d'événements sont attachés au filtre par statut (changement), au champ de recherche (saisie au clavier), et aux deux boutons de réinitialisation (filtres et message vide).

Ouvrier

mes-chantiers:

Cette page permet à l'ouvrier de consulter l'historique de tous les chantiers qu'il a réalisés (candidatures acceptées), avec les détails de chaque mission (fruit, lieu, période, note reçue, commentaire, rémunération perçue).

1. Démarrage de la session et vérification de l'accès

Le PHP démarre la session avec `session_start()`. Il vérifie que l'utilisateur est connecté et qu'il est bien un ouvrier. Si ce n'est pas le cas, l'utilisateur est redirigé vers la page de connexion.

2. Connexion à la base de données

Le PHP inclut le fichier `database.php` pour pouvoir interroger la base de données. Il récupère l'identifiant de l'ouvrier stocké dans la session.

3. Récupération des chantiers réalisés

Le PHP exécute une requête `SELECT` avec plusieurs `JOIN` pour récupérer toutes les candidatures acceptées de l'ouvrier (`decision = 'acceptee'`), avec les détails de chaque offre (fruit, gouvernorat, adresse, dates, prix). Les résultats sont triés par date de fin décroissante (les plus récents d'abord).

4. Calcul des statistiques

Le PHP parcourt en boucle tous les chantiers récupérés. Pour chacun, il additionne la rémunération dans `$total_remuneration` et calcule le nombre de jours travaillés en faisant la différence entre la date de début et la date de fin (`diff->days + 1`). Ces totaux sont affichés dans les cartes statistiques en haut de page.

5. Affichage conditionnel

Le PHP vérifie si le tableau de chantiers est vide ou non. S'il est vide, un message "Aucun chantier effectué" s'affiche avec un bouton pour explorer les offres. S'il contient des chantiers, le PHP génère un tableau HTML avec chaque chantier : offre (fruit + gouvernorat), période, note (avec badge), commentaire et rémunération.

6. Attribution des notes

Pour chaque chantier, le PHP vérifie si une note a été attribuée. Si oui, un badge jaune avec étoile affiche la note sur 10. Sinon, un badge gris "Non noté" s'affiche. La rémunération s'affiche en gras si elle existe, sinon un tiret.

Ouvrier

offres-disponibles:

Cette page permet à l'ouvrier de consulter toutes les offres de récolte disponibles (date limite et date fin dans le futur, places encore libres), de les filtrer par fruit et gouvernorat, de les trier par prix, et de postuler directement à celles qui l'intéressent.

1. Démarrage de la session et vérification de l'accès

Le PHP démarre la session avec `session_start()`. Il vérifie que l'utilisateur est connecté et qu'il est bien un ouvrier. Si ce n'est pas le cas, l'utilisateur est redirigé vers la page de connexion.

2. Connexion à la base de données

Le PHP inclut le fichier `database.php` pour pouvoir interroger la base de données. Il récupère l'identifiant de l'ouvrier stocké dans la session.

3. Traitement de la candidature (postuler)

Le PHP vérifie si l'URL contient `action=postuler` et un identifiant d'offre. Si c'est le cas, il effectue plusieurs vérifications : l'offre existe et sa date limite n'est pas dépassée, l'ouvrier n'a pas déjà postulé à cette offre, et il reste encore des places disponibles. Si toutes les validations sont réussies, il insère une nouvelle candidature avec le statut "en_attente" dans la table `uber_cueillette_candidature`. Un message de succès ou d'erreur est stocké pour être affiché.

4. Récupération des offres disponibles

Le PHP exécute une requête `SELECT` complexe qui récupère toutes les offres valides (date limite et date fin dans le futur, places encore libres) avec le nom du fruit, le gouvernorat, le nombre de places prises, et une information indiquant si l'ouvrier a déjà postulé à chaque offre. Les offres sont triées par date limite croissante (les plus urgentes d'abord).

5. Récupération des listes pour les filtres

Le PHP exécute deux requêtes pour récupérer la liste complète des types de fruits et des gouvernorats. Ces données seront utilisées pour remplir les menus déroulants des filtres.

6. Calcul du nombre total d'offres

Le PHP compte le nombre d'offres disponibles et stocke cette valeur dans `$total_offres` pour l'afficher dans la barre de statistiques.

7. Affichage conditionnel des offres

Le PHP vérifie si la liste des offres est vide. Si c'est le cas, un message "Aucune offre disponible" s'affiche. Sinon, il génère une carte HTML pour chaque offre avec ses informations (fruit, gouvernorat, dates, prix, places restantes, adresse). Si l'ouvrier a déjà postulé, un badge "Postulé" s'affiche ; sinon, un bouton "Postuler" est disponible.

Ouvrier

JAVASCRIPT (VALIDATION)

Filtrer les offres

La fonction `filtrerOffres()` récupère les valeurs des filtres (fruit, gouvernorat) et parcourt toutes les cartes des offres. Elle affiche uniquement celles dont le fruit et le gouvernorat correspondent aux critères sélectionnés.

Trier par prix

Si l'utilisateur choisit un tri par prix croissant ou décroissant, la fonction trie dynamiquement les cartes visibles et les réorganise dans la grille d'affichage en fonction du prix journalier.

Réinitialiser les filtres

La fonction `reinitialiserFiltres()` remet les trois filtres à leurs valeurs par défaut (fruit = tous, gouvernorat = tous, tri = par défaut) et rappelle `filtrerOffres()` pour réafficher toutes les offres.

Initialisation au chargement

Quand la page est chargée, des écouteurs d'événements sont attachés aux trois filtres (changement de fruit, changement de gouvernorat, changement de tri) et aux deux boutons de réinitialisation (filtres et message vide). Le compteur d'offres affichées se met à jour en temps réel et un message "Aucune offre trouvée" apparaît si nécessaire.

Ouvrier

profil:

Cette page permet à l'ouvrier de consulter et modifier ses informations personnelles (nom, prénom, email, description) ainsi que de changer son mot de passe. Les champs CIN et pseudo sont non modifiables pour des raisons de sécurité. La photo de profil est affichée mais n'est pas modifiable depuis cette page

1. Démarrage de la session et vérification de l'accès

Le PHP démarre la session avec `session_start()`. Il vérifie que l'utilisateur est connecté et qu'il est bien un ouvrier. Si ce n'est pas le cas, l'utilisateur est redirigé vers la page de connexion.

2. Connexion à la base de données

Le PHP inclut le fichier `database.php` pour pouvoir interroger la base de données. Il récupère l'identifiant de l'ouvrier stocké dans la session.

3. Récupération des informations de l'ouvrier

Le PHP exécute un `SELECT` pour récupérer toutes les informations de l'ouvrier (nom, prénom, CIN, email, pseudo, description, mot de passe). Si l'ouvrier n'existe pas, la session est détruite et l'utilisateur est redirigé vers la page de connexion.

4. Gestion de l'affichage de la photo

Le PHP vérifie si l'ouvrier a une photo. Si c'est le cas, il convertit les données binaires (BLOB) en base64 pour les afficher dans le navigateur. Sinon, une image par défaut est utilisée.

5. Modification des informations personnelles

Quand l'ouvrier soumet le formulaire de modification, le PHP récupère le nom, prénom, email et description. Il valide que les champs ne sont pas vides et que l'email est valide. Il vérifie également que l'email n'est pas déjà utilisé par un autre ouvrier. Si tout est correct, une requête `UPDATE` met à jour la base de données, et les informations de session sont mises à jour.

6. Changement du mot de passe

Quand l'ouvrier soumet le formulaire de changement de mot de passe, le PHP récupère l'ancien mot de passe, le nouveau et sa confirmation. Il valide que le nouveau mot de passe fait au moins 8 caractères et se termine par `$` ou `#`, et que les deux mots de passe correspondent. Si l'ancien mot de passe (comparé en clair) est correct, la base de données est mise à jour avec le nouveau mot de passe.

7. Affichage des messages

Le PHP stocke les messages de succès ou d'erreur dans les variables `$message` et `$erreur`. Ils sont affichés en haut de la page (vert pour succès, rouge pour erreur).

8. Gestion des erreurs PDO

Le PHP utilise un bloc `try-catch` pour capturer les erreurs de base de données. Si une erreur de doublon (code 1062) se produit, un message spécifique "Cet email est déjà utilisé" s'affiche. Sinon, un message générique d'erreur est affiché.